

It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

Developers routinely attribute the per-request cost of JSON Web Token (JWT) validation to cryptography, and optimise or relocate it on that assumption. We test the assumption empirically in `jsonwebtoken`, a widely used Node.js library. The signature primitive is 2.208 μ s of a 29.92 μ s validation. The dominant component, 18.04 μ s (60%), is a discarded key-parse attempt: a shared secret supplied as a string is first offered to the asymmetric key parser and is resolved only after that parse fails. Its price is set by a transitive dependency, the OpenSSL release bundled with the runtime: issued from C, the same calls cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. In a containerised service, validation adds 384.6 μ s of processor time per request, within 0.8% of the microbenchmark prediction. Of seven JWT libraries in four languages, only this one attempts the parse, so the cost follows a design choice rather than JWT validation as such. The obvious optimisation is unsafe: the parse entered the library with a key-confusion security fix, and removing it naively fails the library’s own tests. We give a narrow fix that passes the full test suite and makes validation 4.15 times faster, and identify pre-parsed keys

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

as the remedy that removes the cost for every input. In our sample of 327 public call sites, none pre-parses the key. We derive guidance for developers, library authors and performance engineers, and release a replication package.

Keywords: Empirical study, performance analysis, software libraries, root-cause analysis, JSON Web Token, OpenSSL

1. Introduction

Performance engineering depends on attributing cost correctly. When a developer believes a cost belongs to one component and it belongs to another, optimisation effort goes to the wrong place, and architectural remedies are adopted for problems that could have been fixed in a few lines. Empirical studies of performance bugs have shown that such misattributions are common, that many performance problems stem from inefficient library usage and avoidable redundant work, and that the fixes are often small once the cause is located (Jin et al., 2012; Selakovic and Pradel, 2016). This paper reports one such case in a security-critical path that runs on every request of many web services: the validation of JSON Web Tokens.

Authentication in cloud-native services is commonly treated as a per-request cryptographic cost, and measurement of layered enforcement has concentrated on the transport and proxy layers where that cryptography runs (Zhu et al., 2023). The assumption is old and has been tested before, one layer down: Coarfa et al. (2006) instrumented TLS web servers and found RSA responsible for only a minority of server cost, the remainder falling to symmetric operations, hashing and protocol handling. Our result is that argument repeated at the application layer, and reaching a sharper version of the same conclusion.

The reasoning being tested is intuitive: validating a JSON Web Token (Jones et al., 2015) means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* (to a sidecar proxy, a gateway, or dedicated hardware) because specialised components do cryptographic work well.

We report that the premise is wrong for `jsonwebtoken`, a widely used JavaScript implementation. A second natural explanation fails as well. Once the signature primitive is shown to be a small share of the cost, the remainder is readily attributed to key material being converted from a string into a key

31 object on every call. Measuring that conversion directly puts it at 0.583 μ s:
32 too small to be the explanation for anything.

33 The actual mechanism is a control-flow accident, shown in Figure 2.
34 Given a secret that is not already a key object, the library attempts
35 `createPublicKey()` first and falls back to `createSecretKey()` in the
36 exception handler. For an HMAC secret the first call cannot succeed. Every
37 validation therefore constructs and discards an OpenSSL parse failure, at
38 18.04 μ s against a total call cost of 29.92 μ s.

39 That 18.04 μ s is the cost of the whole discarded attempt: the work
40 OpenSSL performs before deciding the input is not an asymmetric key, plus
41 the construction and unwinding of the exception that carries that decision
42 back to the library. Section 4.2 separates the two, by reissuing the OpenSSL
43 calls from C and by measuring an exception of the same shape with no
44 OpenSSL call. Where the attempt is expensive it is almost all parse; the
45 exception is a few microseconds.

46 How much the attempt costs depends on which OpenSSL release performs
47 it, by more than an order of magnitude (Section 5). Runtimes carrying an
48 OpenSSL release from 3.2 onward, which include the current Node.js release
49 lines, pay tens of microseconds per validation; runtimes carrying the 3.0
50 series pay hundreds. The 3.0 series is not confined to retired Node.js releases.
51 It is the system OpenSSL of long-term-support distributions still in wide
52 use, among them Ubuntu 22.04 and Debian 12, and a Node.js build that
53 links the system library rather than its bundled copy inherits it; so does any
54 service whose image is pinned to an older base layer, which is common in
55 practice (Zerouali et al., 2019). At either price the discarded attempt is the
56 largest single component of the call, and in every case it is avoidable.

57 The cost is also a security matter, in three ways (Section 8). First, it is
58 paid before authentication: key resolution precedes both the algorithm check
59 and the signature check, so every request carrying a well-formed token pays it,
60 whether or not anyone the service trusts issued that token. Second, it did not
61 arrive through neglect. The resolution block entered the library in version 9.0.0,
62 the release that fixed an RSA-to-HMAC key-confusion vulnerability (Auth0,
63 2022), and the key-type checks that implement that fix read the type of the key
64 object the block produces. Third, it therefore cannot be removed the obvious
65 way: dispatching on the token’s declared algorithm without restricting which
66 key material may take the fast path fails the library’s own key-confusion tests.
67 A performance change to authentication code is a change to its security, and
68 we analyse it as one.

For software engineering the case matters for three reasons beyond the library itself. First, the cost sits in control flow that no profiler label names: a parse whose failure is caught and discarded appears in profiles as OpenSSL work, and the natural reading of that attribution is “cryptography”. Second, its magnitude is set by a transitive dependency that application developers do not choose directly and rarely record, the OpenSSL release bundled with the runtime or base image. Third, the obvious fix is unsafe: the code that costs the time is also the code that makes a security check work, so removing it requires understanding a coupling that is not visible from the lines being changed.

The result is broader than one library in two respects. The design that produces the cost (resolving key material on every call, by way of a parse whose failure serves as a type test) can be named independently of `jsonwebtoken`, and Section 7 shows it predicts which of seven libraries pay. And the reason the cost went unattributed is methodological: the component that sets its price, the bundled OpenSSL release, is one that measurement protocols rarely record at the granularity at which it changes.

Research questions.

RQ1 Where does the per-request cost of token validation actually go?

RQ2 What governs the magnitude of that cost?

RQ3 Does it matter in a running containerised service, and how common is the pattern that avoids it?

RQ4 Is the defect a property of one library, or of a design choice other libraries could share?

RQ5 Can the cost be removed without weakening the library’s key-confusion defence, and who can cause it to be paid?

Contributions. All supported by measurements in the replication package (Katha et al., 2026).

1. A decomposition of the validation call, by two independent methods, locating its dominant component in a discarded key-parse attempt rather than in the signature primitive or in key conversion, and a second decomposition of that attempt into the OpenSSL parse, the V8 exception and the binding between them (Section 4).
2. A causal attribution of its magnitude to the bundled OpenSSL release: the parse reissued from C with no JavaScript engine across 8 identically

- 104 built OpenSSL releases, a runtime pair that crosses engine and library,
105 and a repetition on a second architecture (Section 5).
- 106 3. A measurement in a containerised service across seven arrival rates, and
107 a survey establishing that the avoiding pattern is rare in three samples
108 of public code (Section 6).
- 109 4. A comparison of seven JWT libraries in four languages showing that
110 the cost follows a design choice (resolving key material on every call,
111 of which a failing asymmetric parse used as a type test is the costliest
112 form) rather than JWT validation as such (Section 7).
- 113 5. An analysis of the constraint on removing that component (Section 8).
114 It is reachable before authentication by any well-formed token; it entered
115 the library with a key-confusion fix and is load-bearing for it, so the
116 natural optimisation fails the library’s key-confusion tests; a narrow fix
117 preserves the defence and makes validation 4.15 times faster, but leaves
118 the parse reachable through an attacker-chosen algorithm header; and
119 only a pre-parsed key removes it for every input.
- 120 6. Guidance for application developers, library authors and performance
121 engineers, derived from the findings and summarised in Table 11 (Sec-
122 tion 9).

123 *Organisation.* Section 2 reviews related work and Section 3 the methodology.
124 Sections 4 to 8 answer RQ1 to RQ5 in turn. Section 9 draws implications
125 for practice, Section 10 discusses threats to validity and future work, and
126 Section 11 concludes.

127 2. Background and related work

128 *What a validation call does.* A JSON Web Token carries a header, a payload
129 and a signature over the first two (Jones et al., 2015). Validating one means
130 decoding two segments, resolving key material into a form the cryptographic
131 library accepts, and verifying the signature. The middle step, key resolution,
132 is the subject of this paper, and it is the step most often left implicit when
133 validation cost is discussed.

134 *Where the cost is usually attributed.* Published measurement of layered en-
135 forcement in microservices concentrates on transport and proxy layers. Zhu
136 et al. (2023) decompose service-mesh sidecar overhead and show that the dom-
137 inant contributors depend on configuration and workload rather than being a

138 fixed tax; we reach an analogous conclusion one layer up, for application-level
139 validation. Application-level token validation is itself thinly evidenced, and is
140 commonly reported as a difference between scenario averages, which attributes
141 to validation every difference between two runs. We were unable to locate
142 a study that measures the key-resolution step directly; we state that as an
143 absence we searched for rather than as a claim about the whole literature.

144 *Cryptographic cost has been over-attributed before.* Coarfa et al. (2006) remains
145 the clearest precedent: measuring TLS web servers rather than primitives,
146 they found the public-key operation was a minority of server cost. Work on
147 cryptographic library performance since has largely stayed at the level of
148 the primitive or the protocol exchange (Naser et al., 2020; Sikeridis et al.,
149 2020), with Tuveri and Brumley (2019) a notable exception in examining how
150 a library loads and dispatches implementations rather than how fast those
151 implementations run. Our result concerns neither: it is the cost of getting
152 key material *into* the library.

153 *Performance bugs as a class.* Jin et al. (2012) studied real-world performance
154 bugs across five code bases and found that most follow a small number of
155 recurring patterns (among them uncoordinated and skippable function calls)
156 and that fixes are usually local. Zaman et al. (2012) found, from 400 Firefox
157 and Chrome reports, that performance bugs differ from other bugs in how
158 they are reported, discussed and fixed, in part because reproducing them is
159 hard; Nistor et al. (2013a) report that most are found by code reasoning rather
160 than by observing symptoms. Liu et al. (2014) reach similar conclusions for
161 mobile applications. Detection work targets structural symptoms: redundant
162 traversals with similar memory-access patterns (Nistor et al., 2013b), or
163 predicates that separate slow from fast runs (Song and Lu, 2014). The defect
164 here is of the “skippable call” kind Jin et al. describe, but it has none of the
165 structural signatures those detectors look for: it is one call, executed once
166 per request, whose failure is its only observable effect. It is also the shape
167 that makes a performance bug hard to report: our own upstream report was
168 preceded by a two-year-old report of the symptom with no diagnosed cause
169 (Section 8.6).

170 *JavaScript and Node.js performance.* Selakovic and Pradel (2016) classified
171 performance issues in JavaScript projects and found that inefficient API usage
172 (correct calls made in a costly way) is among the most common root causes,
173 and that the fixes are typically a few lines. Their later profiler (Selakovic et al.,

174 2017) targets precisely the *order* in which a program evaluates conditions, the
175 axis on which this defect lives: a check that almost always fails is evaluated
176 before one that almost always succeeds. Gong et al. (2015) locate code that
177 defeats JIT optimisation; exception-heavy control flow is one such pattern,
178 and Section 4.2 measures what it costs here. Work on the Node.js event loop
179 shows why a per-request synchronous cost matters beyond its magnitude: a
180 single slow handler delays every other client (Davis et al., 2018; Staicu and
181 Pradel, 2018). We make no availability or capacity claim, but the mechanism
182 by which a synchronous per-request cost propagates is the same.

183 *Version and environment variation.* That software performance changes
184 across versions in ways maintainers do not anticipate is established: Kalte-
185 necker et al. (2023) track performance across configurable systems’ release
186 histories, and Chen and Shang (2017) study the code changes that introduce
187 regressions. Laaber and Leitner (2018) assess whether microbenchmark suites
188 detect such changes at all, and Stefan et al. (2017) find performance unit
189 testing rare in practice. Container images make this concrete by pinning a
190 dependency set implicitly: Cito et al. (2017) characterise how the ecosystem
191 uses them, Zerouali et al. (2019) show that images routinely ship outdated
192 packages, and Malka et al. (2026) show that an image tag does not guarantee
193 a reproducible build. For OpenSSL 3 specifically, the maintainers of a major
194 downstream library have stated publicly that parsing and key loading re-
195 gressed substantially against 1.1.1 (Gaynor and Kehrer, 2026); that statement
196 is not peer reviewed and was not produced under a protocol we can assess,
197 and Section 5.5 supplies a controlled measurement for the one path this paper
198 concerns. Section 5 is an instance with a measured cost attached.

199 *Measurement protocol is the thing at stake.* The methodological core of this
200 paper is not new. Georges et al. (2007) established that a single execution
201 of a managed-runtime benchmark is not a measurement, and that intervals
202 must be computed across executions rather than within one; Kalibera and
203 Jones (2013) show how to spend a fixed budget across levels of repetition,
204 which is the reasoning behind our choice of many short processes over few
205 long ones. Mytkowicz et al. (2009) showed that environmental details unre-
206 lated to the code under study can produce effects larger than the one being
207 measured, without any obvious mistake, and Curtsinger and Berger (2013)
208 that memory layout alone can do so. Barrett et al. (2017) found that virtual
209 machines frequently do not reach a steady state, Traini et al. (2023) that

210 developers misjudge when warmup ends, and Costa et al. (2021) catalogue how
211 microbenchmarks go wrong in practice. Mytkowicz et al. (2010) showed that
212 sampling profilers disagree on which method is hot, which is why Section 4
213 uses the profiler only as corroboration of a result obtained by differencing and
214 relies on ordering rather than on its percentages. Our contribution is not to
215 these methods but is an instance of what they warn about. The attribution
216 this paper overturns has exactly the structure Mytkowicz et al. describe: a
217 second variable, the bundled OpenSSL version, moving unrecorded alongside
218 the one under study, at a granularity coarse enough to hide it (Section 5).

219 *Environment as a confounder.* Laaber et al. (2019) quantify how far cloud
220 environments degrade microbenchmark reliability, and Leitner and Cito (2016)
221 characterise performance variation across public infrastructure providers.
222 Their concern is variance between nominally identical environments. Ours
223 is adjacent and, we think, less examined: a *systematic* difference between
224 environments that are not identical and are routinely treated as though they
225 were: two container base images differing in a bundled library version. Laaber
226 et al. are also the reason the second-architecture repetition in Section 5.6 is
227 reported as an ordering check rather than as a replication of magnitudes.

228 *Cryptographic interfaces are hard to use correctly.* The defect reported here
229 is not a cryptographic weakness but a consequence of how a cryptographic
230 interface is invoked, which is a well-documented class. Lazar et al. (2014)
231 found that most of the cryptographic vulnerabilities they studied were misuses
232 of libraries by applications rather than bugs in the libraries. Egele et al. (2013)
233 found cryptographic misuse widespread in deployed applications and Georgiev
234 et al. (2012) found certificate validation broken across a wide range of non-
235 browser software; Nadi et al. (2016) attribute much of it to the interfaces
236 themselves rather than to developer inattention, a position Green and Smith
237 (2016) put directly and Indela et al. (2016) respond to with interface proposals;
238 and Acar et al. (2017) show experimentally that API design materially changes
239 whether developers produce correct code. Detection has been automated by
240 specifying correct usage, as CrySL does (Krüger et al., 2018), and by static
241 analysis at scale (Rahaman et al., 2019). Our finding sits within that line
242 with one difference: the interface here admits a use that is *correct*, and costly
243 only for a reason invisible at the call site. A CrySL-style rule could express
244 “pass a key object, not a string”, but the rule sets we know of encode security
245 properties only, so none would report it.

246 *JWT and JOSE libraries.* Security analyses of JWT implementations consis-
247 tently find key-handling defects. Detering et al. (2017) demonstrated attacks,
248 key confusion among them, against several popular JOSE libraries; Xu et al.
249 (2023) automated the detection of cryptographic vulnerabilities in applica-
250 tions that use JWT; Yang et al. (2026) evaluated 43 JWT libraries across
251 16 programming languages for vulnerabilities; and Shatnawi et al. (2024)
252 surveyed Python JWT libraries for security warnings and adoption. RFC
253 8725 (Sheffer et al., 2020) codifies the resulting practice. All of this work
254 treats a library’s handling of key material as a security surface. Section 7
255 treats the same surface as a cost site, across seven libraries, and Section 8.4
256 shows the two views are coupled: the control flow that costs time in one
257 library is the control flow that enforces its key-type check.

258 *Mining public repositories.* Section 6.5 draws on code hosted publicly, and
259 the hazards of doing so are catalogued by Kalliamvakou et al. (2014), among
260 them that many repositories are not software projects, that activity is not
261 uniformly distributed, and that a search result is not a random sample. Our
262 sampling frame is stated in Section 3.10 with those hazards in view, which is
263 why no population proportion is computed from it.

264 *The cost of exceptions.* That exceptions are expensive in managed runtimes,
265 and that their cost is concentrated in construction rather than in unwinding,
266 is long established: Ogasawara et al. (2006) optimise around exception paths
267 in a Java virtual machine, and Renwick et al. (2019) quantify the cost in a
268 compiled setting. Parravicini and Mueller (2021) provide the closest available
269 cost-attribution baseline for the V8 engine, though on speculation rather than
270 exceptions. Empirical studies of how exceptions are *used* find handlers that
271 discard what they catch to be common: Cabral and Marques (2007) report
272 that handlers in the Java and .NET code they studied rarely attempt recovery,
273 Kery et al. (2016) find that handlers mostly log, print, return or rethrow, and
274 de Pádua and Shang (2017) measure the prevalence of handling anti-patterns,
275 as Weimer and Nacula (2004) did earlier for error-handling mistakes. That
276 literature treats a swallowed exception as a correctness and maintainability
277 smell. The handler at the centre of this paper is a swallowed exception used
278 as control flow on a hot path, and it is a performance defect rather than
279 a correctness one: the fallback it guards is correct. None of these studies
280 measures such a handler’s cost; Section 4.2 does, and finds the exception itself
281 the smaller part.

282 *Key-type confusion.* That a token declaring an HMAC algorithm must not be
283 validated against asymmetric key material is long established; RFC 8725 (Shef-
284 fer et al., 2020) describes the attack and its mitigations. The library studied
285 here has been hardened against this class twice. Versions before 4.2.2 accepted
286 an HMAC token verified with an RSA or EC public key used as the shared
287 secret (CVE-2015-9235) (GitHub Advisory Database, 2015), and versions
288 up to 8.5.1 could still be led to verify forged tokens from RSA to HMAC
289 through their key-retrieval path (CVE-2022-23541), alongside two related
290 defects in key-type and default-algorithm handling (CVE-2022-23539, CVE-
291 2022-23540) (Auth0, 2022); all three were fixed in version 9.0.0. Section 8
292 reports no new attack. It reports that the second hardening introduced the
293 control flow whose cost this paper measures, that the defence depends on
294 that control flow, and that removing the cost naively removes the defence.

295 2.1. What we searched for and did not find

296 Table 1 summarises the positioning. Each cell records only what this
297 section states about the work cited; it is a map of the lines of work this paper
298 connects, not an assessment of them.

299 Three areas bear directly on this result and we were unable to locate
300 verifiable peer-reviewed work in any of them. We state each as an absence
301 we searched for rather than as a claim about the whole literature; each is
302 also, in part, why the misattribution corrected here survived as long as it
303 did. Two adjacent questions (the cost of error construction in V8, and where
304 key-parsing cost lies inside OpenSSL) were equally unstudied, and this paper
305 measures both directly (Sections 4.2 and 5.5).

306 *OpenSSL 3.x performance regressions.* We found no peer-reviewed study
307 of performance regressions across the OpenSSL 3.x series. Evidence exists
308 as a maintainer statement (Gaynor and Kehrer, 2026), as project issues
309 (for instance `openssl/openssl` issues #17627 and #20698, which concern
310 performance of the 3.x provider and decoder machinery) and as mailing-list
311 discussion. None was produced under a measurement protocol we can assess,
312 so Section 5.5 measures the path directly rather than adopting the mechanism
313 those sources propose.

314 *Base image to bundled library to application performance.* We found
315 no academic study tracing that chain. Malka et al. (2026) address build
316 reproducibility, Zerouali et al. (2019) package staleness and Cito et al. (2017)
317 ecosystem practice; none measures how the library version an image pins
318 changes application performance.

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API security*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version as cause*: attributes a cost or defect to the version of a bundled dependency. *Cost-defence coupling*: shows that removing a cost would weaken a security defence. ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API security	Cost	Version as cause	Cost-defence coupling
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	–	–	–
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	–	✓	–	–	–
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	–	–	✓	–	–
Zhu et al. (2023)	–	–	✓	–	–
Jin et al. (2012); Selakovic and Pradel (2016)	–	–	✓	–	–
Kaltenecker et al. (2023); Chen and Shang (2017)	–	–	✓	◦ ^a	–
Zerouali et al. (2019); Cito et al. (2017)	–	–	–	◦ ^b	–
This paper	✓	✓	✓	✓	✓

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured.

319 *Cryptographic API misuse as a performance defect.* The misuse literature
320 cited above treats misuse exclusively as a security problem. We found no
321 work treating a correct-but-costly use of a cryptographic interface as a latency
322 or throughput defect, which is precisely what this paper reports.

323 3. Methodology

324 Figure 1 gives the study’s workflow: the instruments used for each research
325 question and the tables and figures each produces. Every instrument writes
326 to committed run files, from which every number in the paper is generated
327 (Data availability).

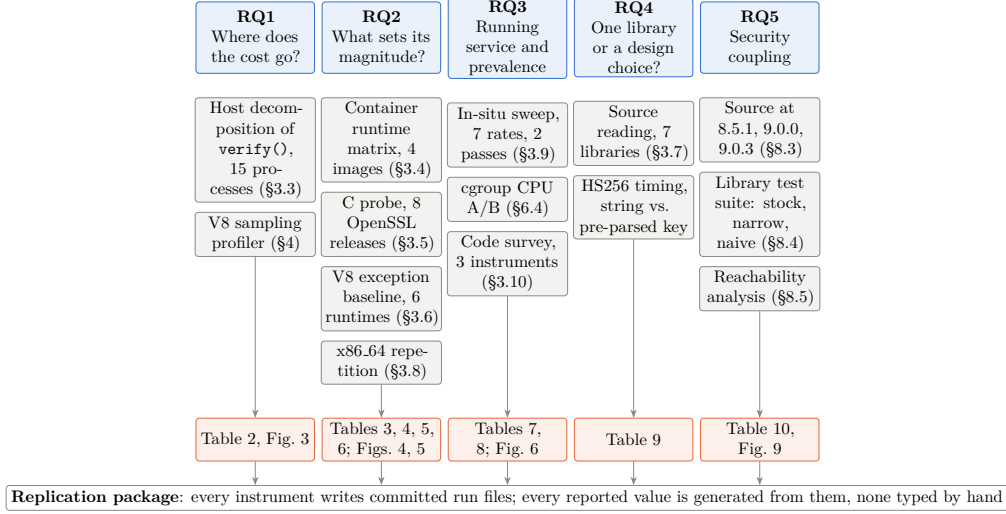


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded attempt for RQ1 (Table 3), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

3.1. Testbed

Unless stated otherwise, measurements were taken on one Apple-silicon machine running macOS 26.5.2 (arm64), referred to throughout as *the host*. The host microbenchmarks of Section 4 ran directly on macOS under Node.js v26.6.0. Container measurements (Sections 5 and 6) ran in the Linux/arm64 virtual machine of Docker Desktop’s built-in single-node Kubernetes cluster (v1.32.2).

The service studied in Section 6 is a Node.js microservice (Express 5.1.0, `jsonwebtoken` 9.0.3) built on the `node:18-alpine` image, run as a single replica with an Istio sidecar (`istio/proxyv2`; the proxy version was not recorded, and its image digest is in the replication package), and reached over TLS through an NGINX ingress controller that is itself injected into the mesh. It verifies HS256 tokens with a shared secret supplied as a string. Load was generated in open loop by k6 v2.2.0 on the same machine, so the load generator, the ingress, the sidecars and the service share one host (Section 3.11). This is a controlled single-node testbed, not a production deployment; we call it the *containerised service*. The x86_64 measurements

345 used the separate machine described in Section 3.8.

346 3.2. Unit of measurement

347 The unit is one operating-system process, not one call. Each process
348 performs 40000 timed iterations of a single condition after a warmup of half
349 that number, and contributes its median. Reported figures are medians of
350 per-process medians, with percentile bootstrap intervals taken over processes.

351 An interval computed over the calls inside one process describes that
352 process’s steady state and is far too narrow: it cannot observe the compilation
353 decisions, inline-cache states and heap layouts that differ between runs of the
354 same code (Georges et al., 2007). Conditions in the decomposition of Section 4
355 were run in 15 separate processes of 40000 calls; conditions in the runtime
356 matrix of Section 5 in 10 processes of 20000 calls. The largest between-process
357 coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring
358 on the `probe_succeeds` condition under `node:26.6.0-bookworm` rather than
359 across the matrix generally.

360 Bootstrap intervals over 15 or 10 units are coarse, and we report them as
361 indicative rather than as calibrated coverage. Where an interval collapses to
362 a point (the timer overhead row of Table 2), it should be read as the clock’s
363 resolution showing through, not as a precise estimate. The per-process values
364 underlying every interval are in the replication package.

365 Conditions are interleaved within each round rather than run in blocks.
366 Under block ordering a thermal excursion or a background process lands
367 entirely on whichever condition is executing and is indistinguishable from an
368 effect; interleaved, it is distributed across conditions and appears as between-
369 round variance. The clock’s own cost is measured in the same process as the
370 condition it accompanies and is reported rather than subtracted: 0.042 μ s
371 against the smallest quantity reported, 0.583 μ s.

372 Differences between conditions are reported throughout as the median
373 of per-round *paired* differences, not as the difference of two medians. The
374 two are not identical and the distinction matters at the magnitudes involved;
375 Section 4.3 and Section 8.4 each rely on it.

376 3.3. Decomposing the call

377 `jwt.verify()` is decomposed into the operations it performs, each timed
378 in isolation under the same protocol: structural decode; the HMAC primitive
379 with a string key and with a pre-parsed key; the symmetric key constructor;

380 and the asymmetric key parse the library attempts first, in both the case
381 where it throws and the case where it succeeds.

382 The library measured throughout is `jsonwebtoken` version 9.0.3, cloned
383 at tag v9.0.3. The code path at issue is shown in Figure 2.

```
120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
secretOrPublicKey === 'string' ? Buffer.from(
secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JsonWebTokenError('secretOrPublicKey is not
valid key material'))
128     }
129   }
130 }
```

Figure 2: Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

384 3.4. *Separating the runtime from the deployment*

385 A host microbenchmark compared against an in-container measurement
386 cannot support an attribution to deployment: those two environments differ
387 in more than deployment, and any of the differences could carry the effect.
388 Here containerisation is held constant and the runtime is varied instead: four
389 container images spanning two OpenSSL series and three V8 major versions,
390 on one machine under one container engine.

391 The pair carrying most of the inference is Node 18 and Node 20: they ship
392 V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock
393 images Node and OpenSSL versions otherwise co-vary, which would leave the
394 attribution underdetermined. Because that pairing carries the attribution,

395 the runtime identity of each image (Node, OpenSSL and V8 version, libc,
396 architecture and image digest) is recorded as a datum and cross-checked
397 against the versions each measurement row carried. That check found 0
398 mismatches across the matrix. Section 5.2 states what this design alone
399 cannot separate, and Sections 3.5 and 3.6 describe the two further designs
400 that do.

401 *3.5. Taking the JavaScript engine out*

402 Runtime pairs can separate engine from library only as far as stock
403 builds allow, so the attribution was tested by removing the engine al-
404 together. We read the Node source at the two release lines measured
405 (`src/crypto/crypto_keys.cc` at v18.20.8; `deps/ncrypto` at v22.23.3,
406 which issues the same sequence) and reproduced in C, call for call, what
407 `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three
408 `PEM_bytes_read_bio` scans for a public key, an RSA public key and a
409 certificate, each under an error-queue mark and each followed by `BIO_reset`;
410 then, since none was recognised, `PEM_read_bio_PrivateKey` with Node's
411 passphrase callback; then the C side of Node's error path, which drains
412 the OpenSSL error queue into strings. The V8 side of that error path is
413 deliberately absent; it is what Section 3.6 measures.

414 The program was compiled once per OpenSSL release with one compiler
415 (GCC 13.3.0) and one set of flags, against 8 releases from 3.0.16 to 3.6.1, each
416 built from source with identical configuration and linked by run path so that
417 no system library could be picked up. Each binary reports the version it
418 actually loaded; rows whose loaded version differed from the one requested
419 would be discarded, and there were 0. The protocol is that of Section 3.2:
420 one condition per process, 15 processes per release and condition, 5000 timed
421 calls each, timed with `clock_gettime`, conditions and releases interleaved
422 within rounds. Besides the full sequence we time its parts separately (the
423 three public-key scans, the private-key parse, the error-string drain) and a
424 *successful* parse of an RSA-2048 public key through the same path.

425 *3.6. Separating the exception from the parse*

426 Inside Node the failed parse is an OpenSSL failure followed by a V8
427 exception: Node constructs an `Error`, attaches `library`, `reason` and `code`
428 properties, throws it through the `createPublicKey()` frame, and the caller
429 discards it. To measure that exception without the parse, a JavaScript
430 benchmark builds the same object (message, property values and error code

431 are captured from a real failure on the runtime being measured, not typed)
432 and throws it from 8 frames below the catch. Variants separate its parts: a
433 plain **Error** thrown and caught in one frame; the Node-shaped error at depth;
434 the same with stack capture disabled; and a Node-internal argument error
435 raised before any OpenSSL call. The real probe is timed alongside, with and
436 without stack capture, so that the stack’s share of it is obtained by paired
437 differencing. All conditions run under official Node release binaries, which
438 bundle their own OpenSSL, on six runtimes (18.20.8, 20.20.2, 22.23.3, 23.11.1,
439 24.21.0 and 26.10.0) at 15 processes of 10000 calls per condition.

440 Combining this with Section 3.5 gives a three-way decomposition of the
441 probe on each runtime: the C cost of the OpenSSL release that runtime
442 bundles, the V8 exception, and a remainder we attribute to the binding
443 between them: argument preparation, key-handle allocation and the C++
444 side of error construction. The three come from different processes and cannot
445 be paired, so the interval on the remainder is obtained by resampling each
446 run’s processes independently.

447 3.7. Other libraries

448 To ask whether the defect is one library’s or a design choice’s, we read the
449 key-handling code of six further libraries, chosen as the most used in each
450 ecosystem: **fast-jwt** and **jose** (JavaScript), **PyJWT** (Python), **golang-jwt**
451 (**Go**), and **nimbus-jose-jwt** and **jjwt** (Java). For each we answer one
452 question from source: given a string HMAC secret, does it attempt to parse
453 it as an asymmetric key? We then time HS256 verification in each with the
454 secret in two forms, under the same one-condition-per-process protocol at 15
455 processes of 20000 calls. “String” is the form each library’s documentation
456 shows for an HMAC secret, used the way its interface is meant to be used;
457 “pre-parsed” is the library’s own key object, built once. Where a library
458 does not accept a string we use its rawest accepted form and say so. The
459 JavaScript libraries are run under two runtimes, one bundling OpenSSL 3.0
460 and one bundling 3.5, because the **jsonwebtoken** penalty depends on that.
461 The penalty is the median of per-round paired differences between the two
462 forms.

463 3.8. A second architecture

464 Tables 2 and 4 were measured on one ARM64 host. Both were repeated
465 with unchanged scripts on an x86.64 machine (Intel(R) Xeon(R) Processor @
466 2.80GHz, four virtual CPUs), including the four container images of Table 4;

Sections 3.5–3.7 were measured on the same machine. That machine is a virtual machine on shared cloud infrastructure, not the dedicated host a replication of magnitudes would require (Laaber et al., 2019; Leitner and Cito, 2016). We use it for what it can support: whether the *ordering* of conditions and runtimes holds on a second instruction set, and ratios between conditions measured interleaved in the same rounds. We do not compare its absolute magnitudes with the ARM64 host’s.

3.9. *In-situ measurement*

The containerised service (Section 3.1) exposes a histogram timing the verification call alone. The instrument is a *paired-scrape difference*: the cumulative histogram is read before and after each arrival-rate step, and the two readings are differenced, which isolates that step’s distribution from the cumulative counters. Means are computed as the difference of the histogram’s `_sum` divided by the difference of its `_count`, so they are exact and do not depend on bucket boundaries; bucket resolution bounds the reported percentiles only.

Seven arrival rates are driven, and the whole sweep is run twice on one process: once in ascending order and once in descending order. We report the *range across the two passes*. This is a range of two, not a confidence interval; no standard error is computed from it, and it is not written as a symmetric tolerance. The opposite-order design also serves as a drift test, which Section 6.2 reports.

One process and two passes is the whole of the in-situ evidence. It is sufficient to establish that the cost is present in the containerised service and of what order; it is not sufficient to characterise the shape of the cost-versus-rate relation, and Section 6.1 is written accordingly.

3.10. *Prevalence: three instruments, three frames*

Prevalence is approached with three instruments that fail in different ways. Their sampling frames differ and are not interchangeable, so no population proportion is computed from them. All three were captured on 2026-09-18.

Population co-occurrence counts. Code-search counts over public repositories, in four cells: two languages by two import forms. The denominator of a cell is files importing the library; the numerator is files that *also mention* the pre-parsing interface. This measures co-occurrence of two strings in one file, not verified usage. The denominators are returned rounded (each is an exact

multiple of 1024), so they are magnitudes rather than counts established file by file.

A classified sample of call sites. Call sites located by search, deduplicated to one file per repository so that no project dominates, with the key argument traced within its file. This is a convenience sample from a relevance-ranked, undocumented ranking; inclusion probabilities are unknown and no weighting back to the population is possible.

A targeted counter-search. Every file co-occurring the library with the pre-parsing interface, fetched in full and adjudicated by hand. This frame is deliberately biased toward finding the pattern (that is its purpose as a negative control) and it is therefore disqualified as a prevalence estimator.

Three reasons preclude a population-level prevalence. The population numerator and the verified numerator measure different things: of 13 automatically flagged call sites, 5 were false positives on hand inspection, so the co-occurrence signal carries an error rate that would be inherited, unmeasured, by any extrapolation. The sample has no inclusion probabilities. And the counter-search is selected on the outcome. Results are therefore reported *in our sample*, or *among repositories that co-occur the interface and call the validator*, and never as a population proportion. This constraint is carried into the abstract and the conclusion.

3.11. Host state is recorded per measurement

A service measured on a host it shares with its own load generator, ingress and sidecars is measured together with whatever else that host is doing. We observed this directly while building the instrument: re-running one configuration with a background indexing process consuming host CPU moved the point at which throughput collapsed by hundreds of requests per second, and both runs are committed in the replication package (Katha et al., 2026). A curve that moves with unrelated host activity does not characterise the service, and this paper therefore makes no capacity claim. Every in-situ window instead carries the host load average observed during it, committed beside the measurement, and Section 6.3 uses it.

4. RQ1: Where the cost goes

4.1. The decomposition

Table 2 gives each operation in the validation path, timed in isolation on the host under the protocol of Section 3, and Figure 3 plots it.

Table 2: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.2).

Operation	Median (μs)	95% interval (μs)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

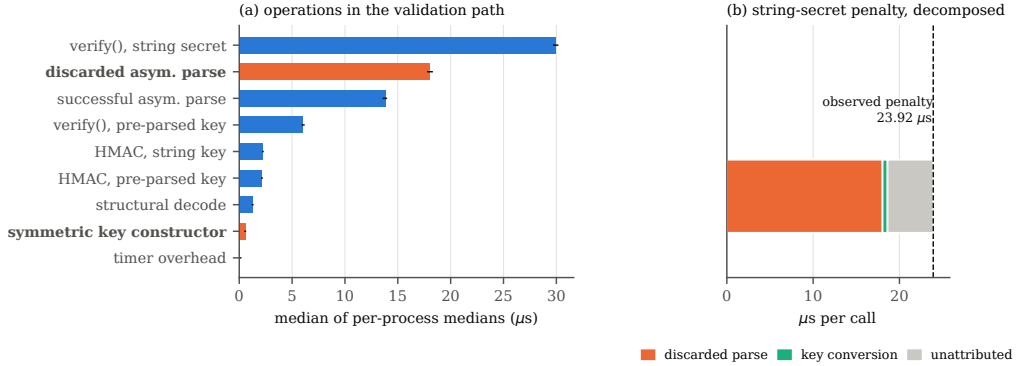


Figure 3: (a) Every operation in Table 2, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.3).

537 The two emphasised rows are the result. The signature primitive costs
538 2.208 μs of a 29.92 μs call. The conversion to which the remainder is most
539 readily attributed costs 0.583 μs . The discarded attempt that precedes that
540 conversion costs 18.04 μs , which is 60% of the whole call and its largest single

541 component.

542 Supplying a pre-parsed key instead of a string reduces the call by 23.92 μ s
543 [23.58, 24.00] over 15 paired rounds.

544 *Key conversion does not explain it.* Once the signature primitive is shown
545 to be small, one explanation for the remainder suggests itself immediately:
546 that the cost lies in converting key material from a string into a key object
547 on every call. It is the natural explanation, and Table 2 refutes it. That
548 conversion costs 0.583 μ s, so the attribution is wrong by roughly the ratio
549 between that figure and 18.04 μ s.

550 A second observation is usually offered in support of the conversion account:
551 an asymmetry between HMAC and RSA costs, read as conversion being dearer
552 than parsing a PEM. It is backwards. A successful asymmetric parse costs
553 13.83 μ s against 0.583 μ s for the conversion. The real asymmetry is that the
554 RSA path’s parse *succeeds*, making its cost useful work, while the HMAC
555 path’s fails, making it waste.

556 4.2. What the 18.04 μ s is made of

557 The 18.04 μ s is the cost of the discarded attempt taken whole. It contains
558 at least two things: the work OpenSSL performs before concluding that the
559 input is not an asymmetric key, and the construction and unwinding of the
560 exception that returns that conclusion to the library. Table 2 already bounds
561 the split: a *successful* asymmetric parse (the same entry point, reaching a
562 conclusion and throwing nothing) costs 13.83 μ s, the same order as the failing
563 one, so the exception cannot be most of it. Table 3 replaces the bound with a
564 decomposition, combining the C measurement of the same OpenSSL release
565 (Section 3.5) with the exception baseline (Section 3.6) on six official runtimes.

566 Three things follow. First, on every runtime bundling OpenSSL 3.0 the
567 C parse alone matches the probe to within a few percent: 101% of it under
568 Node 18.20.8 and 103% under Node 23.11.1, the excess over 100% being the
569 difference between our build of the release and Node’s. There the discarded
570 attempt *is* the OpenSSL parse, and it would cost the same with no JavaScript
571 engine present. Second, the exception is small and nearly constant across
572 runtimes: the Node-shaped error costs 4.781 μ s under Node 18.20.8 and
573 3.421 μ s under Node 26.10.0 (1.2% and 19.0% of the respective probes), and
574 roughly half of it is stack capture (with capture disabled, 2.204 and 1.930 μ s).
575 Disabling stack capture for the real probe saves 5.397 μ s and 1.767 μ s on
576 the same two runtimes, the same order as the synthetic figure. Third, on

Table 3: The discarded attempt decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

577 runtimes bundling OpenSSL 3.5, where the parse is cheap, the three parts
578 are of comparable size: under Node 26.10.0 the parse is 47% of the probe,
579 the exception 19.0% and the binding 34.1%.

580 The component is therefore a discarded *parse attempt*, not an exception
581 cost. Where it is expensive the exception is about a hundredth of it, and even
582 where it is cheap the exception is a fifth. These runtimes are Linux builds on
583 x86_64; the ARM64 host figure of Table 2 was not itself decomposed this way,
584 and its 18.04 μs is of the same order as the OpenSSL 3.5 rows here, not the
585 OpenSSL 3.0 rows.

586 4.3. The parts do not account for all of it

587 If the penalty for supplying a string were exactly the discarded attempt
588 plus the conversion, the two would sum to it. They do not, quite:

	observed penalty	23.92 μs
589	discarded parse + conversion	18.62 μs
	residual	5.292 μs

590 All three quantities are medians of per-round paired differences, per
591 Section 3.2. The residual’s interval, [4.916, 5.501], excludes zero. The two

592 measured parts therefore account for 77.9% and not more; the remainder
593 lies in the library’s own type and claim checking along the non-pre-parsed
594 path. We report it rather than absorbing it into the headline, because a
595 decomposition whose parts are declared to sum exactly to the whole has
596 usually not been checked.

597 The pre-parsed path carries a smaller unexplained remainder of the same
598 kind, and we record it for consistency rather than leaving the asymmetry
599 unremarked: `verify()` with a pre-parsed key costs 6.042 μ s against 1.292 μ s of
600 structural decode plus 2.166 μ s of HMAC, leaving roughly 2.58 μ s unattributed.
601 Neither residual was measured directly; both are the subject of conditions we
602 did not run, and both are stated as unattributed rather than assigned.

603 4.4. Confirmation without differencing

604 The decomposition reaches its conclusion by subtracting conditions, the
605 same inference style this paper criticises elsewhere. It is therefore checked
606 against V8’s sampling profiler, which attributes self time to named frames
607 with no arithmetic on our part. At 50000 iterations per run and 3 runs per
608 environment, matched so the percentages are comparable, the asymmetric key
609 parse accounts for a median 66.2% of self time on the host (range 65.3–67.4
610 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under
611 Node 26. The frame does not appear among the top frames of the pre-parsed
612 path in any environment.

613 Two methods, one answer. We rely on the ordering and the dominance
614 rather than on the exact percentages: sampling attribution depends on rate
615 and on what else the process is doing.

616 5. RQ2: What governs the magnitude

617 Table 4 varies the runtime with containerisation held constant, and Figure 4
618 plots the two columns that matter.

619 5.1. Engine major version does not distinguish the two slow runtimes

620 Node 18 and Node 20 ship V8 10 and 11 (different major versions of the
621 engine) and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μ s.
622 Node 26 ships OpenSSL 3.5 and pays 22.94 μ s on the same libc. Within the
623 OpenSSL 3.0 series, then, changing the engine’s major version does not move
624 the quantity; had Node 20 been fast, the attribution would have been to the
625 engine.

Table 4: The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. [‡]The host row is from Table 2 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10).

Runtime	OpenSSL	V8	Disc. parse (μ s)	Pre-parsed (μ s)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host [‡]	3.6.3 [†]	14	18.04	6.042

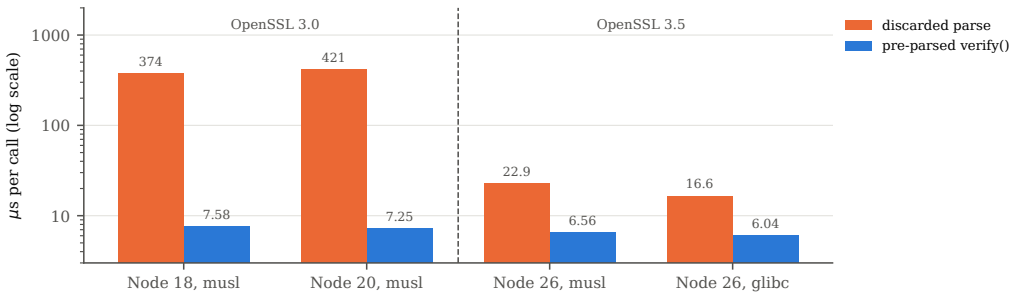


Figure 4: The four container rows of Table 4 on a logarithmic axis. The discarded parse spans $25.36\times$ across images while the pre-parsed call spans $1.26\times$. The dashed rule marks the OpenSSL series boundary. Within this matrix it is not a controlled boundary, because the two fast images also carry a newer engine and a different build (Section 5.2); Section 5.2 controls it. The uncontainerised host row is omitted because its comparison with the container rows is confounded.

626 5.2. What this design cannot separate

627 That argument establishes a negative within one OpenSSL series. It does
628 not establish the positive attribution for the change between series, and we
629 state the limitation rather than leaving it implicit.

630 The fast rows are Node 26. They carry OpenSSL 3.5 and V8 14 and every
631 other change in that build. OpenSSL version is therefore perfectly confounded
632 with engine version and with build configuration across the 3.0–3.5 boundary.
633 Nothing in this matrix can separate them. The absence of variation between
634 V8 10 and V8 11 does not license the inference that V8 14 is also irrelevant;

635 that is a different comparison, and we did not make it.

636 What the matrix supports on its own is the following, and no more: the
637 magnitude co-varies with the bundled OpenSSL version across four images;
638 the two slow images differ in engine major version and do not differ in
639 magnitude; and the two libc variants of one image differ by less than the
640 difference between series. Separating OpenSSL from the engine requires a
641 crossed design: a runtime pairing a newer engine with an older OpenSSL, or
642 the parse measured outside Node entirely. Section 5.5 reports both, and it is
643 on them, not on this matrix, that the causal wording used in this paper rests.

644 5.3. *The effect is confined to key parsing*

645 Across the four container rows, pre-parsed validation spans $1.26\times$, the
646 HMAC primitive $1.39\times$ and structural decode $1.35\times$. The discarded parse
647 spans $25.36\times$. The successful parse moves with it ($11.74\times$), so this is the cost
648 of key parsing in general rather than anything peculiar to the failure path.
649 The failure is simply the case in which an application pays that cost on every
650 request for a result it discards.

651 5.4. *What this design does not isolate*

652 A reader may notice that the uncontainerised host row is of the same
653 order as the Node 26 container row. That comparison is confounded and we
654 draw no conclusion about containerisation from it: the host runs a different
655 operating system from the containers, so operating system, C library and Node
656 build all change alongside the container boundary. Isolating containerisation
657 would require a Linux-native baseline, which we did not obtain. The claims
658 above concern only the four container rows, among which containerisation is
659 constant.

660 5.5. *Caused by OpenSSL: the parse without an engine*

661 Table 5 removes Node entirely: the OpenSSL calls `createPublicKey()`
662 makes for an HMAC secret, issued from C against 8 OpenSSL releases built
663 identically, and Figure 5 plots its first and last columns.

664 In plain C the failure costs $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on
665 3.5.8, a ratio of 46.69 [46.16, 46.90]. Nothing else differs between those two
666 measurements: the program, the compiler, the flags, the machine and the
667 protocol are the same. The magnitude is therefore *caused* by the OpenSSL
668 release. That is a claim about this code path in these releases; it says nothing
669 about OpenSSL performance generally.

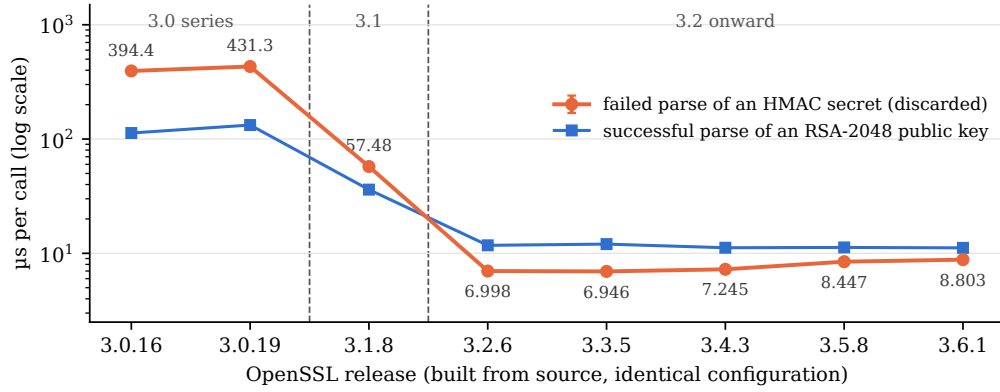


Figure 5: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration (Table 5, column *Full*), beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

Table 5: The failed asymmetric parse issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one compiler and one set of flags (x86_64), in μ s. Medians of per-process medians over 15 processes of 5000 calls. *Full*: everything `createPublicKey()` asks of OpenSSL for an HMAC secret, including the error-string drain; 95% intervals for it are in the replication package and are within 0.9% of the median for every release. Its parts follow. *Successful parse*: an RSA-2048 public key through the same path, for comparison.

OpenSSL	Full	Private-key parse	Public-key scans	Error strings	Successful parse
3.0.16	394.4	390.9	2.287	0.796	112.9
3.0.19	431.3	424.0	2.231	0.869	132.6
3.1.8	57.48	54.84	2.196	0.497	36.09
3.2.6	6.998	4.487	2.057	0.459	11.75
3.3.5	6.946	4.465	2.074	0.474	12.04
3.4.3	7.245	4.554	2.098	0.500	11.19
3.5.8	8.447	5.757	2.051	0.483	11.26
3.6.1	8.803	5.985	2.234	0.472	11.16

670 The table also says where in OpenSSL the cost lives and when it went
671 away. Almost all of it is the private-key parse (99% of the full sequence on

Table 6: The runtime matrix of Table 4 on two architectures, same images and scripts, in μs . ARM64: 10 processes of 20000 calls. x86_64, a shared cloud virtual machine: 10 processes of 20000 calls. Absolute values are not comparable across the two machines.

Image	OpenSSL	Discarded parse		Pre-parsed <code>verify()</code>	
		ARM64	x86_64	ARM64	x86_64
<code>node:18.20.8-alpine</code>	3.0.16	373.79	607.1	7.58	12.66
<code>node:20-alpine</code>	3.0.19	420.62	556.3	7.25	11.89
<code>node:26.6.0-alpine</code>	3.5.7	22.94	42.13	6.56	10.94
<code>node:26.6.0-bookworm</code>	3.5.7	16.58	19.91	6.04	9.701

672 3.0.16), the step at which OpenSSL 3 hands the input to its provider-based
673 decoders; the three public-key scans reject the input at the PEM header and
674 are cheap in every release, and the error strings cost under a microsecond.
675 The fall came in two steps: to 57.48 μs in 3.1.8, and to 6.998 μs in 3.2.6, after
676 which it is flat to within a few microseconds through 3.6.1. We measured the
677 steps, not the source changes that produced them; identifying those would
678 take a bisection of the OpenSSL history that we have not done. A successful
679 parse of real key material fell with it, which is why the effect is confined to
680 key parsing in general rather than to the failure path.

681 *Engine and library crossed.* The official runtimes of Section 3.6 contain the
682 pairing stock container images lack: Node 22.23.3 ships V8 12.4.254.21 with
683 OpenSSL 3.5.8, while Node 23.11.1 ships the *newer* V8 12.9.202.28 with the
684 *older* OpenSSL 3.0.16. The probe costs 18.98 μs on the first and 383.5 μs
685 on the second. The Node 18/20 pair of Table 4 showed that a change of
686 engine at a fixed OpenSSL series leaves the cost where it was; this pair shows
687 that a newer engine does not rescue an older OpenSSL. Together with the C
688 measurement, the engine is excluded from both directions.

689 5.6. A second architecture

690 Table 6 repeats the runtime matrix, with the same images and scripts,
691 on x86_64 (Section 3.8). Absolute values are not compared across the two
692 machines; the question is whether the ordering holds.

693 The property the attribution rests on (every OpenSSL 3.0 image slower
694 than every OpenSSL 3.5 image) holds on both architectures. The strict rank
695 order does not: the two OpenSSL 3.0 images, Node 18 and Node 20, exchange
696 places between ARM64 and x86_64. Their difference is small against the gap

697 between series on either machine, and no claim in this paper depends on
698 their relative order. On x86_64 the discarded parse spans $30.50\times$ across the
699 images while pre-parsed validation spans $1.31\times$, against $25.36\times$ and $1.26\times$ on
700 ARM64. The host decomposition of Table 2 was repeated too (Node 22.22.2,
701 OpenSSL 3.5.5): the discarded parse is $19.42\ \mu\text{s}$ of a $36.48\ \mu\text{s}$ call (53%), the
702 symmetric constructor $1.261\ \mu\text{s}$ and the HMAC primitive $3.105\ \mu\text{s}$, so the
703 component ordering of Table 2 holds there as well. The full x86_64 host table
704 is in the replication package.

705 6. RQ3: The containerised service, and prevalence

706 6.1. In-situ cost across arrival rates

707 The containerised service was driven at seven arrival rates, each measured
708 by the paired-scrape difference of Section 3.9, with the whole sweep run
709 ascending and then descending on one process. Table 7 reports the range
710 across the two passes at each rate for the configuration the service deploys, a
711 string secret, and Figure 6 plots it.

Table 7: In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μs)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

712 One observation follows, and it is the one this section claims: the cost is
713 present in the containerised service and is far larger than the isolated host
714 figure of Table 2, consistent with Section 5 in that the service’s runtime is
715 the one carrying the older OpenSSL.

716 Two things visible in Table 7 need care, and we set them out explicitly.

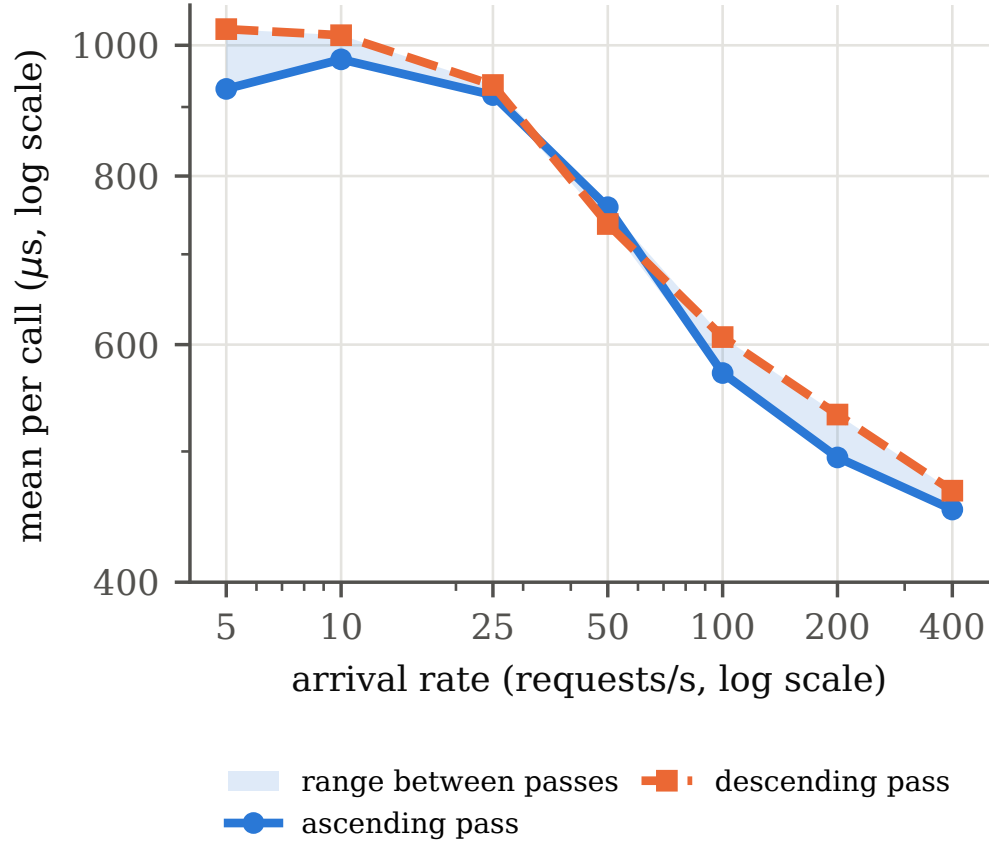


Figure 6: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged: Section 6.2 reports that they differ systematically, and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

717 *Wall-clock exceeds the prediction; processor time does not.* The service run-
718 time’s discarded parse costs 373.79 μs (Table 4), and the remainder of the
719 validation path is of order 7.58 μs , giving an expected total near 381 μs . The
720 wall-clock means observed across the sweep are 453–1028 μs , exceeding that
721 by 71–647 μs . Contention inflates a duration rather than deflating it, so these
722 are upper bounds, and Section 6.4 shows where the excess is not: measured
723 as processor time, the validation call costs within 0.8% of the prediction. The
724 excess is therefore time the call spent off the processor on a shared host, not
725 additional work. We did not decompose that off-processor time further, and
726 no claim below depends on it.

727 *The decline with arrival rate is not interpreted.* Cost falls monotonically from
728 928.24 μs at 5 requests/s to 452.82 μs at 400 requests/s. This is the opposite
729 of what contention alone would produce, and several mechanisms unrelated
730 to validation would produce it: a fixed per-window overhead amortising over
731 more calls, processor frequency scaling and idle-state exit latency at low rates,
732 cache and branch-predictor residency, or continued tier-up in the engine. We
733 did not run the conditions that would distinguish them. We therefore report
734 the numbers and do not claim that validation cost depends on arrival rate;
735 establishing that would require varying window duration at fixed rate, pinning
736 the frequency governor, and extending warmup at low rates, none of which
737 we did.

738 6.2. The two passes differ systematically

739 Sweeping up and then down is not only replication; it detects warm-up,
740 thermal and hysteresis effects. If the difference between passes were noise,
741 its sign would alternate across rate points. It does not. In the string-secret
742 sweep the descending pass is slower at 6 of 7 rate points; in the pre-parsed
743 sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven
744 points is not what noise produces.

745 This has a direct consequence for how the ranges in Table 7 should be read.
746 A systematic component means the two passes are not independent samples of
747 a common value, so the range between them understates uncertainty further
748 than a range of two already does. We therefore report ranges and draw no
749 inference that depends on their width. It is also a methodological finding in
750 its own right: a single-direction sweep would have produced a curve carrying
751 a consistent bias with no way to detect it.

752 *6.3. Host load, not the condition, drives the outliers*

753 Agreement between passes varies considerably across windows. We dis-
754 tinguish *relative* disagreement, the gap between passes as a fraction of their
755 mean, from *absolute* disagreement, the gap in microseconds; the two behave
756 differently and the distinction carries this subsection.

757 The widest relative disagreement in the dataset and the tightest occur at
758 the same arrival rate in different sweeps. The wide one is the highest-load
759 window in its own sweep on the descending pass, though not on the ascending
760 pass, whose maximum load falls at a different rate. The tight one, at the
761 same arrival rate in another sweep, sits at a lower load on both passes and
762 its two passes agree to within $0.02\ \mu\text{s}$. Relative disagreement therefore does
763 not track the condition under measurement.

764 We considered two alternative explanations for the relative asymmetry and
765 reject both. A small-mean effect would require absolute disagreement to be
766 roughly constant, so that its relative size grew only as means shrank; absolute
767 disagreement instead varies over three orders of magnitude and is largest
768 where the means are largest, which is the opposite pattern. A measurement
769 floor would require the affected values to sit at the instrument’s resolution;
770 they do not, and in any case the means reported here are computed from the
771 histogram’s sum and count rather than from its buckets, so bucket resolution
772 cannot bound them.

773 The operative consequence is that in-situ figures are reported with the
774 host load recorded during their window, and comparisons between windows
775 at different loads are not treated as comparisons between conditions.

776 *6.4. Corroboration: the cost is processor time, and it matches the prediction*

777 An in-situ figure of this magnitude has two readings. Either it is work the
778 service performs, or it is time the request spends waiting, queued behind other
779 requests, descheduled by a contended host, or inflated by the instrument.
780 We separate them with an A/B of the same service at one arrival rate, with
781 validation enabled in one window and disabled in the next, on the same
782 pod running the same image, and read the processor time the application
783 container was charged from its cgroup accounting rather than from a clock.
784 Table 8 reports it.

785 Enabling validation adds $384.6\ \mu\text{s}$ of processor time per call. The runtime
786 matrix predicts $381\ \mu\text{s}$ for the same work on the same runtime (Section 6.1);
787 the two agree to within 0.8%. They are obtained by instruments that share
788 no code: cgroup CPU accounting on the service’s pod, and a single-process

Table 8: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

microbenchmark in an isolated container. A request that queues or is de-scheduled accrues wall-clock time but no CPU, so this figure cannot contain queueing delay.

Three observations weigh against reading the difference as an artefact of the two windows. The sidecar, in the same pod under the same contention, did not rise with the application: 53.40 millicores disabled against 49.13 enabled. The one-second CPU samples of the two arms do not overlap at all (disabled never exceeded 100.577 millicores in 127 samples, enabled never fell below 114.963 in 127), which separates the arms without a distributional assumption. And event-loop lag is unchanged, so neither arm was near saturation. The two windows are two observations, not a sample: the difference carries no interval and we compute none.

6.5. How common is the avoiding pattern, in our sample?

Three instruments, three frames, reported separately as Section 3.10 requires. All counts were captured on 2026-09-18. None of the figures below is a population proportion.

Population co-occurrence. Of 790528 public JavaScript files importing the library through `require`, 29 also mention the pre-parsing interface; for JavaScript through `import` the figures are 524288 and 38; for TypeScript through `require`, 8288 and 0; and for TypeScript through `import`, 333824

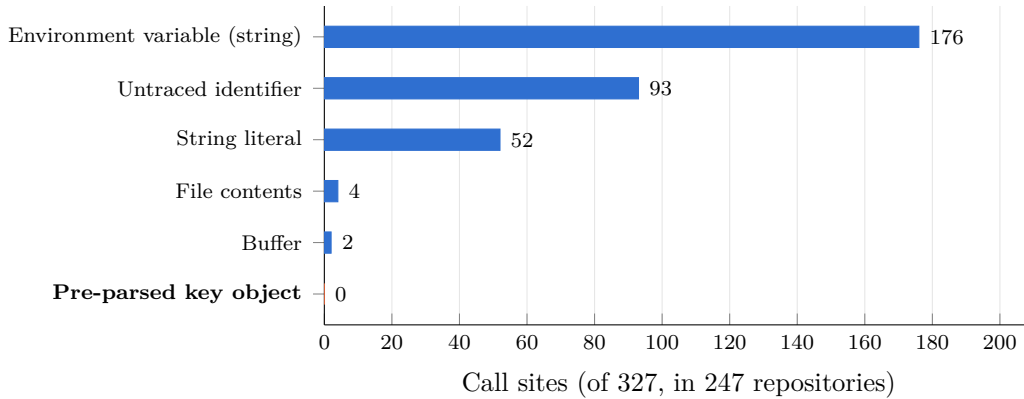


Figure 7: Where the secret passed to the validator comes from, for the 327 call sites classified in our sample. No call site passes a pre-parsed key object, the one form that avoids the discarded parse for every input. Independently, 309 of the 327 call sites (94%) pass no algorithm restriction. Counts describe the sample, not the population (Section 3.10).

809 and 101. Cells are not pooled into a rate: a file using both import forms
 810 would be counted twice, so their sum 1656928 is an upper bound on the union
 811 and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files
 812 mentioning the interface. These are search-engine estimates of file counts over
 813 public repositories only, they measure co-occurrence rather than use, and the
 814 denominators are returned *rounded* (each is an exact multiple of 1024), so
 815 they should be read as magnitudes rather than as counts established file by
 816 file.

817 *Classified call sites.* Of 327 call sites classified across 247 distinct repositories
 818 in our sample, 0 pass a pre-parsed key. Environment variables account for
 819 176 of those call sites, untraced identifiers and member expressions 93, string
 820 literals 52, file contents 4 and buffers 2 (Figure 7). No file in this corpus
 821 contains the pre-parsing interface at all, in any spelling, so the zero reflects
 822 the pattern’s absence from the sample rather than a classifier that cannot
 823 see it. Incidentally, 309 of 327 call sites (94%) pass no algorithm restriction,
 824 which Section 8.4 returns to.

825 Matching only the conventional `jwt.verify()` spelling would filter the
 826 corpus toward one import idiom, so call sites are found by resolving the local
 827 binding of the import, covering arbitrary aliases, namespace imports, destruc-
 828 tured and renamed imports, and direct calls on the `require` expression. That

829 binding resolution contributes 12 of the call sites above that the conventional
830 spelling misses, of which 0 pass a pre-parsed key.

831 *Targeted counter-search.* A sample can only show a pattern is rare if it
832 could have found it. Every file co-occurring the library with the pre-parsing
833 interface was therefore fetched: 191 files, of which 120 contain a validation
834 call, spanning 112 distinct repositories. An automated pass flagged 13 call
835 sites as passing a pre-parsed key and all were then read by hand: 5 are false
836 positives (projects defining their own helper function of the same name, one
837 of which returns a string), 1 could not be resolved within its file, and 7 are
838 genuine, across 6 distinct projects.

839 *The rate, with its denominator.* Among the 112 repositories that both refer-
840 ence the pre-parsing interface and call the validator, 6 use it: 5.36%. That
841 denominator is the most favourable available, since those projects demonstra-
842 bly know the interface exists, and it is not the population. Within this frame
843 the pattern is rare rather than absent, and it is rare even among projects
844 already aware of it. We do not extrapolate beyond the frame.

845 7. RQ4: One library, or a design choice?

846 A cost found in one library is an observation about that library. It becomes
847 a statement about a class of defect only if the mechanism can be named
848 independently of the library and predicts which other libraries have it. The
849 mechanism here can be named: *resolving key material of unknown type by*
850 *attempting the asymmetric parse first, and treating its failure as the signal*
851 *that the material is a shared secret.* Table 9 tests the prediction against six
852 further libraries.

853 *What the source says.* Only `jsonwebtoken` attempts the asymmetric parse.
854 The others resolve the ambiguity in one of three ways, none of which involves
855 a parse that can fail on a shared secret. *Inspect, then dispatch:* `fast-jwt`
856 examines the text of the key (whether it carries a PEM header or is a JSON
857 Web Key) and calls exactly one constructor, skipping even that inspection
858 when every permitted algorithm is HMAC; `PyJWT` likewise inspects the text,
859 and rejects PEM- or SSH-looking material offered as an HMAC key rather
860 than trying to parse it. *Refuse the ambiguous input:* `jose` does not accept
861 a string for an HMAC key, and `golang-jwt` accepts only a byte slice for
862 HMAC verification, returning a key-type error for anything else. *Type the*

Table 9: HS256 verification across seven JWT libraries in four languages (x86_64), in μs . *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.7). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

863 *key*: jjwt requires a **SecretKey** for HMAC and a **PublicKey** for signatures,
864 so the distinction is made by the Java type system; nimbus-jose-jwt offers
865 a constructor that takes a string and converts its bytes without parsing them.
866 Refusing a string does not by itself remove per-call work, however: jose,
867 handed raw bytes, imports them into a WebCrypto key on every call.

868 *What the measurements say.* Of the seven libraries, two pay for being handed
869 their raw form rather than a key object, in the sense that the paired interval
870 on the penalty excludes zero; for the other five it does not, and their point
871 estimates are within a microsecond of zero. The two are jsonwebtoken
872 and jose, and they pay for different reasons. jsonwebtoken performs the
873 failing asymmetric parse: its penalty is 465.9 μs under OpenSSL 3.0.19 and
874 24.56 μs under OpenSSL 3.5.8, a string-to-pre-parsed ratio of 46.55 and 3.48
875 respectively (Figure 8). jose parses nothing that can fail, but imports the
876 raw bytes into a WebCrypto key on every call: 42.66 μs and 24.80 μs under
877 the same two runtimes. Both penalties grow on OpenSSL 3.0, so the OpenSSL
878 release sets the price of per-call key handling generally; but the failing parse
879 is the costliest form of it, about 11 times jose’s on OpenSSL 3.0. Two of the
880 libraries that pay nothing resolve the key once, when a verifier or parser is
881 built (fast-jwt, jjwt), so a raw secret costs nothing per call by construction;
882 that is itself the design point.

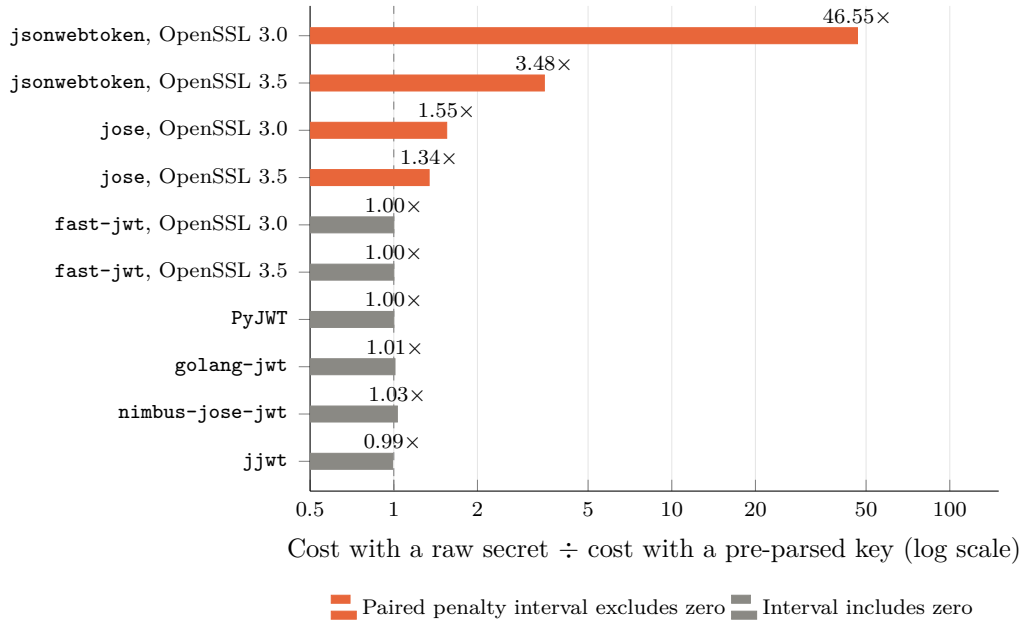


Figure 8: How much each library pays for being handed its raw secret rather than its own pre-parsed key object: the ratio of the two costs in Table 9, on a log scale. A ratio of one (dashed line) means no per-call penalty. Orange: the paired 95% interval on the penalty excludes zero. Only `jsonwebtoken` performs the failing asymmetric parse, and only it pays by more than a small factor; `jose` pays for re-importing raw bytes on every call.

883 *What this establishes.* The libraries without the problem matter as much as
 884 the ones with it. All seven validate the same kind of token with the same
 885 primitive (against the same OpenSSL, in the JavaScript cases) and five pay
 886 nothing for a raw secret. The cost therefore follows a design choice, resolving
 887 key material on every call, and not JSON Web Token validation, HMAC or
 888 OpenSSL as such: OpenSSL 3.0 sets the price, but only a library that makes
 889 that choice is charged it, and one that makes it by way of a failing parse is
 890 charged the most. The failing parse is exactly what the order-of-evaluations
 891 profiler of Selakovic et al. (2017) is designed to find, and the alternatives
 892 above are the fixes that line of work suggests: reorder the test, make it cheap,
 893 or move it off the per-call path. The comparison does not estimate how many
 894 libraries make the choice; it shows that making it is what matters.

895 8. RQ5: Removing the cost safely

896 Sections 4–7 treat the discarded attempt as a cost. This section asks
897 who can cause it, why it is there, and what removing it does to the library’s
898 defences. Its evidence is the library’s source at the releases named, the
899 library’s own test suite, and the measurements already reported; it introduces
900 no new timing condition, and where a statement rests on reading code rather
901 than on measurement we say so.

902 8.1. *Threat model*

903 The adversary is a remote client that can place arbitrary bytes where the
904 service expects a bearer token, including every field of the token header, and
905 knows which library the service uses. It does not know the shared secret
906 and cannot produce a valid HMAC. The service verifies HS256 tokens with a
907 shared secret supplied as a string, the form the library’s documentation shows
908 and the form every call site in our sample uses (Section 6.5). The asset of
909 interest is key-type separation: a token declaring an HMAC algorithm must
910 never be verified against asymmetric key material, nor the reverse (Sheffer
911 et al., 2020). Out of scope are compromise of the secret, defects elsewhere
912 in the library, and availability. We did not drive adversarial traffic at the
913 containerised service and make no availability or capacity claim; the question
914 here is which inputs reach the discarded attempt, not how many a service
915 can absorb.

916 8.2. *The attempt precedes authentication*

917 Figure 9 shows the order in which `verify()` performs its checks at
918 tag v9.0.3. Reaching the key-resolution block of Figure 2 requires only
919 that the token have three dot-separated segments, a header and payload that
920 decode as JSON, and a non-empty signature segment (lines 59–118). Nothing
921 on that path depends on the secret. The permitted-algorithm list is consulted
922 afterwards, at line 144; the key-type checks at lines 148–160; the signature at
923 line 165.

924 Two consequences follow. A token that will be rejected pays the discarded
925 attempt in full: 18.04 μ s on the host, and 373.79 μ s on the runtime the studied
926 service deploys (Table 4). And restricting permitted algorithms, the defence
927 RFC 8725 recommends and which 94% of the call sites in our sample omit,
928 does not avoid it, because the restriction is enforced after resolution. The cost
929 is therefore under the control of any client that can reach the endpoint, not

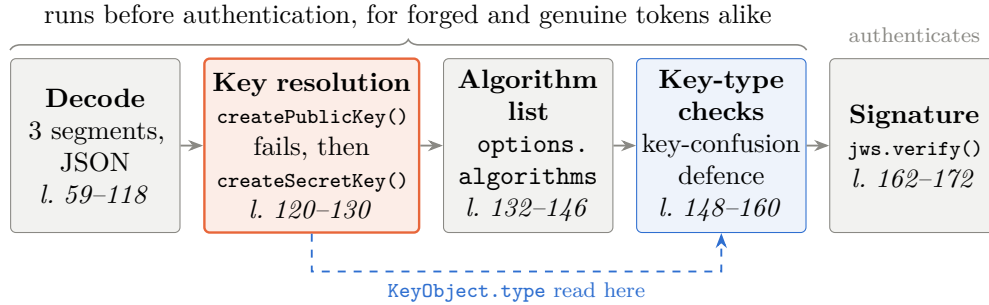


Figure 9: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded attempt sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

930 only of the service’s legitimate load. That is a statement about reachability,
 931 established from the source; its magnitude is the one measured in Sections 4–6.

932 8.3. The attempt arrived with a key-confusion fix

933 We compared `verify.js` in the published packages of versions 8.5.1, 9.0.0
 934 and 9.0.3. In 8.5.1 the secret is passed to the signature routine as given, with
 935 no key-object resolution. The resolution block of Figure 2 first appears in 9.0.0
 936 and is unchanged, apart from whitespace, through 9.0.3. Version 9.0.0 is the
 937 release that fixed CVE-2022-23541 and CVE-2022-23539 (Auth0, 2022), and
 938 the checks implementing those fixes (the comparison of the declared algorithm
 939 with the key’s type at lines 148–152 and the validation of asymmetric key
 940 types at lines 154–160) operate on the key object the resolution block produces.
 941 The attempt is thus not an oversight in otherwise untouched code. It is part of
 942 a security fix, and its price is set by whichever OpenSSL release a deployment
 943 happens to bundle (Section 5).

944 This is consistent with the one earlier report of the symptom: HS256
 945 verification in the 9.x series measured 34 times slower than in 8.5.1, on a
 946 Node release bundling OpenSSL 3.0 (Section 8.6). We did not time version
 947 8.5.1 ourselves, and state the connection as consistent with the report rather
 948 than as established by it.

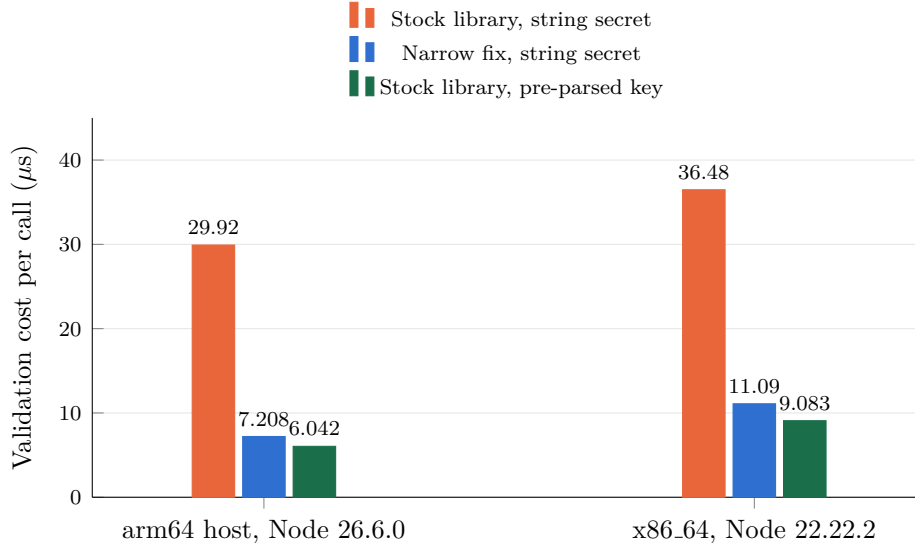


Figure 10: Per-call HS256 validation cost under the stock library with a string secret, under the narrow fix with the same string secret, and under the stock library with a pre-parsed key, on both machines (medians of per-process medians). The narrow fix recovers most of the gap without any change to calling code; only the pre-parsed key removes the attempt for every input (Section 8.5).

949 8.4. *A fix, and a constraint on it*

950 The fix is to dispatch on the token’s declared algorithm before attempting
 951 the asymmetric parse, so that an HMAC token with an ordinary string secret
 952 is resolved directly. Applied, this takes the call from 29.92 μs to 7.208 μs , a
 953 factor of 4.15, and to within 1.208 μs of supplying a pre-parsed key, with no
 954 change in calling code. That residual is the median of the per-round paired
 955 differences, not the difference of the two medians; the two are not identical
 956 and the former is reported throughout.

957 Figure 10 shows the same comparison on the x86_64 machine of Section 3.8,
 958 where the narrow fix takes the call from 36.48 μs to 11.09 μs against 9.083 μs
 959 with a pre-parsed key.

960 We tested it against the library’s own suite rather than only against
 961 assertions of our own. Cloned at tag v9.0.3, the patch applies cleanly and the
 962 suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

963 These figures are from the host. The practical benefit is largest where the
 964 discarded parse is largest, of order 373.79 μs on the runtime the containerised
 965 service uses (Table 4), and we did not measure the patched library on the

966 container runtimes. The reported factor of 4.15 is therefore a lower bound
967 on the improvement available in the containerised service’s runtime, and we
968 state it as a host result rather than as a general one.

969 *The constraint, which is the more important half.* The discarded attempt is
970 not purely wasteful. The *success* of that parse on asymmetric key material
971 is what produces a key object whose type a later check depends on, and
972 that check is what prevents symmetric and asymmetric material from being
973 interchanged (Sheffer et al., 2020). A change that dispatches on the declared
974 algorithm while relaxing which key material may take the fast path alters
975 which inputs reach that check.

976 The fix must therefore be narrow: the fast path is restricted to string
977 secrets that cannot be asymmetric key material, so anything that might
978 be resolves exactly as it does today. Removing that material restriction
979 (the form of the optimisation we argue against) produces 2 failures in the
980 library’s own suite, both in its key-confusion tests (“should not allow HMAC
981 verification with an RSA key in PEM format” and, under “signing with pub
982 key as symmetric”, “should not verify”). All of these outcomes (the suite
983 unchanged under the narrow patch, two key-confusion failures without its
984 guard) reproduce on the x86_64 machine of Section 3.8 under a different Node
985 and OpenSSL release (Katha et al., 2026).

986 *The existing suite already guards the property.* No new regression test is
987 needed: the library’s existing suite constrains the change, and a contributor
988 proposing the unrestricted optimisation would be caught by tests they are
989 required to run. The finding is therefore not a missing test but a hidden
990 coupling: the cost documented in Section 4 cannot be removed by the obvious
991 route, and the reason is a property the maintainers already test for but which
992 is not evident from the code being changed.

993 The independent defence against this class of confusion is for the caller to
994 restrict permitted algorithms (Sheffer et al., 2020), and 94% of the call sites
995 surveyed in Section 6.5 do not.

996 8.5. What the narrow fix leaves reachable

997 The narrow fix selects its fast path on the token’s declared algorithm,
998 and the declared algorithm is chosen by whoever constructs the token. For a
999 token declaring an HMAC algorithm and a plain string secret the parse is
1000 skipped. For a token declaring any other algorithm the patch falls through

Table 10: Which tokens reach the discarded key-parse attempt, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.4. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

to the original resolution, so `createPublicKey()` is attempted on the shared secret and fails exactly as it does today; the token is then rejected by the algorithm list or the key-type check, but only after the parse. Table 10 sets out which inputs reach the attempt under each configuration. The narrow fix removes the cost from legitimate traffic, not from traffic an adversary constructs. Only a pre-parsed key object removes it for every input, because the resolution block is skipped entirely when the configured key is already a `KeyObject` (line 120).

This follows from the code and was not separately timed: we did not measure the patched library on tokens declaring a non-HMAC algorithm, and the per-call cost such a token incurs is that of the discarded attempt measured in Sections 4 and 5. A variant that dispatched on the *configured* algorithms rather than the declared one would close the gap for callers who restrict permitted algorithms to HMAC, since tokens declaring anything else are rejected regardless; it is the choice `fast-jwt` makes (Section 7). We did not implement or test it, it would help only the minority of callers who restrict algorithms, and it would have to keep the same guard on key material.

8.6. Reported upstream

Before reporting, we searched the library’s issue tracker for earlier reports. One exists: an issue opened in April 2024¹ reports that HS256 verification in the 9.x series is 34 times slower than in 8.5.1, on a Node release that bundles OpenSSL 3.0. It diagnoses no cause and, at the time of writing,

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

has no maintainer response. That is the symptom this paper explains, observed independently two years earlier by a user who could not see its source. We reported the mechanism, with a minimal reproduction, per-runtime measurements and the OpenSSL finding, as a separate issue², and a pull request implementing the narrow form of the fix, with its guard, is open against it. At the time of writing neither has received a maintainer response.

9. Implications for practice

Table 11 maps each finding to the evidence behind it and to the stakeholder who can act on it. The paragraphs that follow expand on each recommendation.

Pre-parse key material once, at startup. On the host this removes 23.92 μ s per validation [23.58, 24.00]; on the runtime the service actually deploys, where the discarded parse costs 373.79 μ s, it removes far more. It requires no architectural change, no new component and no movement of a trust boundary, and it is the only remedy in this paper that removes the attempt for every input, including tokens an adversary constructs (Table 10). On a current runtime bundling OpenSSL 3.5 the saving is smaller (24.56 μ s per validation in Table 9), but it is not negligible beside the 347.3 μ s of processor time the containerised service spends on a request with validation disabled (Table 8). The two figures come from different machines and are compared as orders of magnitude only.

Treat the base image as part of the measurement. The same code, the same library and the same architecture differ by 25.36 $\times$ on this path across the container images measured, because they pin different OpenSSL versions. A performance figure quoted without the base image it was obtained on is not reproducible, and a regression appearing after a base-image bump may have nothing to do with the application. Section 5.5 shows that on this path the OpenSSL release is the cause; but the practical point would hold even if it were not, since it is enough that the image changes the number.

For library authors: do not use a failing parse as a type test. Section 7 found three designs that avoid the cost (inspect the material and dispatch, refuse

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

Table 11: Summary of findings, evidence and implications, by research question and by the stakeholder who can act on each.

RQ	Finding	Evidence	Implication (who acts)
RQ1	The signature is 2.208 μ s of a 29.92 μ s call; a discarded key-parse attempt is 18.04 μ s (60%).	Table 2, Fig. 3; profiler	Profile before relocating a “cryptographic” cost; attribution by intuition was wrong twice here (<i>performance engineers</i>).
RQ2	The parse costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, issued from C.	Tables 4, 5; Fig. 5	Record the base image and bundled OpenSSL with every performance figure; treat base-image bumps as performance changes (<i>developers, DevOps</i>).
RQ3	In a containerised service validation adds 384.6 μ s of CPU per request; no sampled call site pre-parses its key.	Tables 7, 8; Fig. 7	Pre-parse key material once at startup: a one-line change (<i>application developers</i>).
RQ4	Of seven libraries in four languages only one attempts the parse; five pay nothing for a raw secret.	Table 9, Fig. 8	Do not use a failing parse as a type test; resolve keys once, inspect then dispatch, or type the key (<i>library authors</i>).
RQ5	The parse is load-bearing for a key-confusion defence; the naive optimisation fails 2 library tests, the narrow fix passes all 511 and is 4.15 $\times$ faster.	Table 10; Figs. 9, 10	Review performance changes to verification code as security changes; run the key-confusion tests (<i>maintainers, reviewers</i>).

1054 ambiguous input at the interface, or type the key) and all three ship in
 1055 maintained libraries today. What they share is that resolution happens once,
 1056 or not at all, rather than on every call. The design that pays most is the one
 1057 that asks an expensive parser to answer a cheap question on every request.

1058 *Pin the algorithm.* Independently of anything here, callers should restrict
 1059 permitted algorithms (Sheffer et al., 2020). Most of the call sites we surveyed
 1060 do not, which leaves them relying entirely on library-internal key typing for a
 1061 property they could assert directly in one argument. Pinning does not remove

the cost measured here, which is incurred before the restriction is checked (Section 8.2); it is an independent defence, not a substitute for pre-parsing.

For security reviewers: review performance changes to verification as security changes. The cheapest-looking optimisation of this path fails the library’s key-confusion tests, and the property it breaks is not evident from the lines being changed (Section 8.4). Conversely, the security fix that introduced the path carries a cost that varies by more than an order of magnitude with the bundled OpenSSL release (Sections 5 and 8.3). Changes to token verification should be run against the library’s key-confusion tests and timed on the oldest cryptographic library a deployment may bundle.

9.1. Relocation is not the first remedy

A cost believed to be cryptographic invites relocation to a sidecar or gateway. The cost measured here is not cryptographic and is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for. We implemented that relocation for the service studied here; applying a mesh validation policy duplicates verification rather than moving it, and making it effective requires the application to trust a proxy-supplied claims header, a known anti-pattern that needs a filter stripping client-supplied copies. We report that only as a design note: it carries no measurement, and the configuration, the observation and the inconclusive automated probe are documented in the replication package (Katha et al., 2026).

10. Threats to validity and future work

The limitations below are grouped by what they bear on: the measurements, the deployment evidence, the survey of public code, and the scope of the claims. Each states what the paper does not claim as a result.

10.1. Measurement

Two architectures, one of them shared. The primary measurements come from a single ARM64 host. Section 5.6 repeats them on x86_64, and there the split between OpenSSL series and the component ordering of the host decomposition hold. But the x86_64 machine is a virtual machine on shared cloud infrastructure, whose absolute timings are known to be unreliable (Laaber et al., 2019); we therefore take only orderings and ratios between interleaved

1095 conditions from it, and no magnitude in this paper should be read as holding
1096 on x86_64 hardware in general. A repetition on a dedicated x86_64 host, which
1097 the replication package scripts in one command, would turn the ordering
1098 check into a replication of magnitudes. The C probe, the exception baseline
1099 and the cross-library comparison were measured only on the x86_64 machine.
1100 Their conclusions rest on ratios between conditions measured in the same
1101 rounds, which is the kind of quantity a shared machine can support, and
1102 the effects they report are far larger than the machine’s noise: the median
1103 between-process coefficient of variation is 1.5%, 2.9% and 2.8% in the three
1104 runs. The largest values (28.0%, 27.7% and 20.2%) occur on sub-microsecond
1105 or JIT-dominated conditions that carry no claim, and medians of per-process
1106 medians are insensitive to them. The C probe needs no JavaScript runtime
1107 and the scripts rebuild every OpenSSL release from source, so repeating it on
1108 the ARM64 host or a dedicated x86_64 machine is one command.

1109 *The C probe reproduces Node’s calls, not Node’s build.* Section 5.5 issues the
1110 OpenSSL calls Node issues, as read from the Node source, against OpenSSL
1111 releases we built. Node builds its bundled OpenSSL with its own configuration,
1112 so the C figures approximate rather than reproduce the parse cost inside a
1113 given runtime, and the binding column of Table 3 inherits that approximation;
1114 it is slightly negative on two OpenSSL 3.0 runtimes for that reason. The causal
1115 claim does not inherit it: it compares releases under one build configuration
1116 with nothing else varying.

1117 *The exception baseline is a reconstruction.* Node constructs the error from
1118 C++ through V8’s API; our baseline constructs an error of the same shape,
1119 message and properties from JavaScript at a comparable stack depth. The
1120 two paths share V8’s error construction and stack capture, and the paired
1121 stack-capture measurement on the real probe agrees in order of magnitude
1122 with the synthetic one, but they are not the same code path. The ARM64
1123 host figure of Table 2 was not itself decomposed; Table 3 decomposes the
1124 same component on six other runtimes.

1125 *The headline host figure has no per-row OpenSSL version.* The host decom-
1126 position of Section 4 predates the per-measurement OpenSSL field. The
1127 version in force is known from the run timestamp and a note recorded at the
1128 time: OpenSSL 3.6.3, which a package upgrade later that day moved to 3.6.4
1129 under the same Node binary. We report it as inferred, marked as such in
1130 Table 4, rather than as recorded. Three measurements that do carry their

1131 version bound how much this matters. The same probe costs 18.01 μ s under
1132 Node 26.10.0 with OpenSSL 3.5.8, 19.42 μ s in the x86_64 host repetition with
1133 OpenSSL 3.5.5, and 16.58 μ s in the Node 26 container with OpenSSL 3.5.7:
1134 all of the same order as 18.04 μ s, and all from OpenSSL releases after the
1135 step of Table 5. The headline figure is therefore representative of a post-3.2
1136 OpenSSL, which is all the paper uses it for; it would be wrong by more than
1137 an order of magnitude only if the host had run an OpenSSL 3.0 or 3.1 release,
1138 which the record excludes.

1139 10.2. Deployment measurement

1140 *Ranges of two, with a systematic component.* The in-situ dispersion is the
1141 range across an ascending and a descending pass. It is not a confidence
1142 interval, and Section 6.2 shows the two passes differ systematically rather
1143 than randomly, so they are not independent samples of a common value and
1144 the range understates uncertainty further. No inference in this paper depends
1145 on the width of those ranges.

1146 *One in-situ process, two passes.* The rate sweep rests on one process sweeping
1147 seven rates twice. That is sufficient for the claim made (the cost is present
1148 in the containerised service and of this order) and insufficient for any claim
1149 about the shape of the cost-versus-rate relation, which Section 6.1 accordingly
1150 does not make. The magnitude claim does not rest on the sweep alone: the
1151 A/B of Section 6.4 is a separate pair of windows, measured with a separate
1152 instrument, and it agrees with the microbenchmark prediction on processor
1153 time. The wall-clock excess over that prediction is attributed to time off
1154 the processor and not decomposed further; the decline with arrival rate has
1155 several candidate explanations unrelated to validation that we did not test
1156 (Section 6.1).

1157 *Host load, and the confound that motivated recording it.* A throughput curve
1158 measured on a host shared with its own load generator moves with unrelated
1159 activity on that host, as Section 3.11 describes, which is why no capacity claim
1160 appears in this paper. Every in-situ window here carries the load average
1161 observed during it, and Section 6.3 shows that window-to-window agreement
1162 tracks that load rather than the condition. Contention inflates a duration
1163 rather than deflating it, so in-situ figures are upper bounds; but comparisons
1164 between windows at different loads are not comparisons between conditions,
1165 and we do not treat them as such.

1166 *The A/B is two windows at unequal load.* The two arms of Table 8 are
1167 consecutive windows rather than interleaved rounds, and their host load
1168 is known only at the instant each began. That bars comparing them on
1169 wall-clock time, and we do not. It does not bar the CPU comparison the
1170 section rests on: contention inflates elapsed time, whereas a container’s own
1171 cgroup accounting charges it only the processor time it used. Attaching the
1172 per-second host sampler to a re-run of both arms, interleaved, would remove
1173 the limitation; we did not.

1174 *10.3. Survey of public code*

1175 *Sampling frames are not interchangeable.* The three prevalence instruments of
1176 Section 3.10 have different units and different selection mechanisms, and none
1177 supports a population proportion. The co-occurrence counts measure two
1178 strings appearing in one file, with a false-positive rate established at 5 of 13 on
1179 hand inspection. The classified sample has unknown inclusion probabilities, a
1180 limitation of repository search noted more generally by Kalliamvakou et al.
1181 (2014). The counter-search is selected on the outcome.

1182 *The matcher’s coverage.* Call sites are found by resolving the local binding
1183 of the import, which covers aliases, namespace imports, destructured and
1184 renamed imports and direct calls on the require expression. It does not resolve
1185 bindings across files, so a project that constructs a pre-parsed key in one file
1186 and validates in another is invisible to this analysis. This is the strongest
1187 threat to Section 6.5; within-file tracing found no such case, which is weak
1188 evidence against it.

1189 *One rater.* The 13 automatically flagged call sites were read and classified
1190 by the first author, and the authors are not disinterested: the paper’s claim
1191 is stronger the more of them are false positives, and 5 were classified that
1192 way. There was no second rater and no inter-rater agreement to report. The
1193 per-case verdicts, the evidence for each and the hashes of the files they were
1194 read from are published so the adjudication can be repeated, which is weaker
1195 than having repeated it.

1196 *10.4. Scope*

1197 *Token caching is not in the comparison.* A service that caches verified tokens
1198 pays this cost once per token rather than once per request. We did not
1199 measure such a configuration, so the per-request figures here do not describe
1200 a caching deployment.

1201 *The security analysis is from source.* Section 8 establishes reachability and
1202 provenance by reading `verify.js` at the releases named and by running the
1203 library’s own tests; it does not drive forged tokens at the containerised service,
1204 time the patched library on tokens declaring a non-HMAC algorithm, or
1205 time version 8.5.1. Its statements are therefore about which inputs reach the
1206 attempt and when the code arrived, not about throughput under attack, and
1207 the paper makes no availability claim.

1208 *Seven libraries, one version each.* Section 7 covers seven libraries in four
1209 languages, chosen as the most used in each ecosystem, each at one release.
1210 Other libraries, and other releases of these, may resolve keys differently; each
1211 classification is from source at the version measured. The comparison supports
1212 the claim that the cost follows a design choice rather than JWT validation; it
1213 does not estimate how many libraries make that choice.

1214 10.5. Future work

1215 Four directions follow from the limitations above.

1216 *Security.* Section 8 establishes which inputs reach the discarded attempt; it
1217 does not measure what an adversary can do with it. Driving forged tokens at
1218 the containerised service, under the stock library and under the narrow fix,
1219 would turn that reachability statement into an availability measurement. The
1220 configured-algorithm variant of the fix (Section 8.5) should be implemented
1221 and run against the library’s key-confusion tests, and a fix that removes the
1222 attempt for every input without requiring callers to pre-parse keys remains
1223 to be designed and proposed upstream.

1224 *Cause.* The C probe locates the fall in two steps, at 3.1 and 3.2 (Section 5.5).
1225 Bisecting the OpenSSL history between those releases would identify the
1226 changes responsible, and timing version 8.5.1 of the library would test the
1227 connection to the earlier upstream report directly (Section 8.3).

1228 *Generality.* Seven libraries at one version each show that the cost follows a
1229 design choice; they do not estimate how often the choice is made. A systematic
1230 survey of JWT and JOSE libraries, and of other authentication paths that
1231 resolve key material on every call, would. Resolving call sites across files and
1232 adding a second rater would strengthen the prevalence evidence of Section 6.5.

Deployment. A dedicated x86_64 repetition, a Linux-native baseline for containerisation, interleaved A/B windows with per-second host sampling, a controlled study of the decline with arrival rate, and a deployment that caches verified tokens would each close one limitation stated above. The scripts for the first are already in the replication package.

11. Conclusion

The per-request cost of token validation in this library is real, is worth attending to, and is not the signature: the primitive is $2.208\ \mu\text{s}$ of a $29.92\ \mu\text{s}$ call. It is not key conversion either, which costs $0.583\ \mu\text{s}$. It is a parse attempted in the wrong order, whose failure is constructed and thrown away on every request at $18.04\ \mu\text{s}$, 60% of the call. Where that attempt is expensive it is almost entirely OpenSSL work; the exception that ends it costs a few microseconds.

It is also a security property of the path. The attempt runs before the algorithm check and the signature, so any well-formed token can cause it; it entered the library with a key-confusion fix and that fix depends on it, so removing it naively fails the library’s key-confusion tests; and the narrow fix that keeps the defence still leaves it reachable through an attacker-chosen algorithm header. Pre-parsing the key once, at startup, is both the fastest configuration and the only one no input can push onto the slow path.

Its price is set by the OpenSSL release a base image happens to pin, varying $25.36\times$ across runtimes while the rest of the validation path varies by $1.26\times$, and set causally, not only by association: the same calls from C cost $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, and a newer engine paired with an older OpenSSL is the slow runtime, not the fast one.

It is a design choice, not a property of JWT: of six other libraries in four languages none attempts the parse, five pay nothing for a raw secret, and the one that re-imports its key on every call pays an order of magnitude less. The symptom had been reported upstream two years before this study without a diagnosis; we have reported the mechanism.

The pattern that avoids the cost is rare in our sample: among 112 repositories that already reference the pre-parsing interface and call the validator, 6 use it. No population proportion follows.

The explanation that presents itself before the measurement, that the dominant cost is key conversion, is refuted by measuring the conversion. It has the shape this paper’s methodology is built to avoid: a cost attributed to the

variable under study while a second variable moves alongside it, unrecorded at the granularity at which it could change. The bundled OpenSSL version in Section 5 is the same hazard caught in time: the quantity varies $25.36\times$ with a component most measurement protocols record once per run, if at all. The lesson is not that one should measure in deployment. It is narrower: record the identity of the component whose behaviour is being measured, on every measurement, at the granularity at which it can change.

CRedit authorship contribution statement

Jannatul Ferdous Katha: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:** Supervision, Project administration, Writing – review & editing. **Dip Nandi:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The data supporting the results of this study are openly available in the replication package (Katha et al., 2026), archived at Zenodo under <https://doi.org/10.5281/zenodo.23011153> and maintained in a public GitHub repository.³ It contains all measurements, the instrumentation, the analysis code and the scripts that generate every numeric value and every figure

³<https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->

1296 in this paper, including the C program and the script that builds each
1297 OpenSSL release, the exception baseline, and the cross-library harnesses in
1298 four languages with their pinned dependency manifests. Every number is
1299 generated from a file under the run or survey directories; none is entered by
1300 hand. The survey corpus is third-party source and is not redistributed; a
1301 manifest pinning every file examined by content hash stands in for it, covering
1302 502 files across 457 repositories. Content hashes resolved for all of them;
1303 the repository HEAD commit resolved for 501 of 502, one repository having
1304 become unavailable between fetch and manifest. The content hash is the
1305 authoritative pin and is complete.

1306 **Ethics and responsible disclosure**

1307 This study measures software behaviour and publicly available source
1308 code; it involves no human participants and no personal data. No attack
1309 was run against any system the authors do not control. The key-confusion
1310 behaviour discussed in Section 8 is an existing, publicly documented attack
1311 class tested only through the library’s own test suite; the upstream report
1312 describes the constraint at the level of mechanism and does not include an
1313 exploitation recipe.

1314 **Declaration of generative AI and AI-assisted technologies in the** 1315 **manuscript preparation process**

1316 During the preparation of this manuscript, generative AI tools were used to
1317 improve language and readability. The authors reviewed all content following
1318 such use and take full responsibility for the contents of the manuscript.

1319 **Acknowledgements**

1320 The authors acknowledge ELITE Research Lab for providing the compu-
1321 tational resources and support necessary for this research.

1322 **References**

1323 Acar, Y., Backes, M., Fahl, S., Garfinkel, S., Kim, D., Mazurek, M.L., Stransky,
1324 C., 2017. Comparing the usability of cryptographic APIs, in: 2017 IEEE
1325 Symposium on Security and Privacy (SP). doi:10.1109/SP.2017.52.

1326 Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security
1327 update for jsonwebtoken. Auth0 security bulletin, 21 December 2022,
1328 [https://auth0.com/docs/secure/security-guidance/security-bul-](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
1329 [letins/2022-12-21-jsonwebtoken](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken). Accessed 2026-09-28.

1330 Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Vir-
1331 tual machine warmup blows hot and cold. *Proceedings of the ACM on*
1332 *Programming Languages* 1, 52:1–52:27. doi:10.1145/3133876.

1333 Cabral, B., Marques, P., 2007. Exception handling: A field study in Java
1334 and .NET, in: *ECOOP 2007 – Object-Oriented Programming*, Springer.
1335 pp. 151–175. doi:10.1007/978-3-540-73589-2_8.

1336 Chen, J., Shang, W., 2017. An exploratory study of performance regression
1337 introducing code changes, in: *2017 IEEE International Conference on*
1338 *Software Maintenance and Evolution (ICSME)*. doi:10.1109/ICSME.2017
1339 .13.

1340 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C.,
1341 2017. An empirical analysis of the Docker container ecosystem on GitHub,
1342 in: *2017 IEEE/ACM 14th International Conference on Mining Software*
1343 *Repositories (MSR)*. doi:10.1109/MSR.2017.67.

1344 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS
1345 web servers. *ACM Transactions on Computer Systems* 24. doi:10.1145/11
1346 24153.1124155.

1347 Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong
1348 with my benchmark results? Studying bad practices in JMH benchmarks.
1349 *IEEE Transactions on Software Engineering* 47, 1452–1467. doi:10.1109/
1350 TSE.2019.2925345.

1351 Curtsinger, C., Berger, E.D., 2013. STABILIZER: Statistically sound perfor-
1352 mance evaluation, in: *Proceedings of the 18th International Conference on*
1353 *Architectural Support for Programming Languages and Operating Systems*
1354 *(ASPLOS ’13)*, pp. 219–228. doi:10.1145/2451116.2451141.

1355 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
1356 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
1357 *Proceedings of the 27th USENIX Security Symposium*, pp. 343–359.

1358 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017.
1359 On the (in-)security of JavaScript object signing and encryption, in: Pro-
1360 ceedings of the 1st Reversing and Offensive-oriented Trends Symposium
1361 (ROOTS '17). doi:10.1145/3150376.3150379.

1362 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical
1363 study of cryptographic misuse in Android applications, in: Proceedings of
1364 the 2013 ACM SIGSAC Conference on Computer and Communications
1365 Security (CCS '13). doi:10.1145/2508859.2516693.

1366 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography.
1367 pyca/cryptography project statement, [https://cryptography.io/en/1](https://cryptography.io/en/latest/statements/state-of-openssl/)
1368 [atest/statements/state-of-openssl/](https://cryptography.io/en/latest/statements/state-of-openssl/). Maintainer statement, not peer
1369 reviewed; accessed 2026-09-24.

1370 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java
1371 performance evaluation, in: Proceedings of the 22nd Annual ACM SIG-
1372 PLAN Conference on Object-Oriented Programming Systems, Languages
1373 and Applications (OOPSLA '07). doi:10.1145/1297027.1297033.

1374 Georgiev, M., Iyengar, S., Jana, S., Anubhai, R., Boneh, D., Shmatikov, V.,
1375 2012. The most dangerous code in the world: Validating SSL certificates
1376 in non-browser software, in: Proceedings of the 2012 ACM Conference on
1377 Computer and Communications Security (CCS '12). doi:10.1145/238219
1378 6.2382204.

1379 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-
1380 2015-9235, GHSA-c7hr-j4mj-j2w6). [https://github.com/advisories/GH](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6)
1381 [SA-c7hr-j4mj-j2w6](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6). Accessed 2026-09-28.

1382 Gong, L., Pradel, M., Sen, K., 2015. JITProf: Pinpointing JIT-unfriendly
1383 JavaScript code, in: Proceedings of the 10th Joint Meeting on Foundations
1384 of Software Engineering (ESEC/FSE '15). doi:10.1145/2786805.2786831.

1385 Green, M., Smith, M., 2016. Developers are not the enemy! the need for usable
1386 security APIs. IEEE Security & Privacy 14. doi:10.1109/MSP.2016.111.

1387 Indela, S., Kulkarni, M., Nayak, K., Dumitras, T., 2016. Helping Johnny
1388 encrypt: Toward semantic interfaces for cryptographic frameworks, in:
1389 Proceedings of the 2016 ACM International Symposium on New Ideas, New

1390 Paradigms, and Reflections on Programming and Software (Onward! 2016).
1391 doi:10.1145/2986012.2986024.

1392 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and
1393 detecting real-world performance bugs, in: Proceedings of the 33rd ACM
1394 SIGPLAN Conference on Programming Language Design and Implementa-
1395 tion (PLDI '12), pp. 77–88. doi:10.1145/2254064.2254075.

1396 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT).
1397 Request for Comments RFC 7519. Internet Engineering Task Force. doi:10
1398 .17487/RFC7519.

1399 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
1400 Proceedings of the 2013 International Symposium on Memory Management
1401 (ISMM '13). doi:10.1145/2464157.2464160.

1402 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian,
1403 D., 2014. The promises and perils of mining GitHub, in: Proceedings of
1404 the 11th Working Conference on Mining Software Repositories (MSR 2014).
1405 doi:10.1145/2597073.2597074.

1406 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
1407 Performance evolution of configurable software systems: An empirical study.
1408 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

1409 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replica-
1410 tion package: a pre-authentication key parse dominates JWT validation in
1411 jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: <https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->.

1414 Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer
1415 practices for locally handling exceptions, in: Proceedings of the 13th
1416 International Conference on Mining Software Repositories (MSR '16), pp.
1417 484–487. doi:10.1145/2901739.2903497.

1418 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An
1419 extensible approach to validating the correct usage of cryptographic APIs,
1420 in: 32nd European Conference on Object-Oriented Programming (ECOOP
1421 2018), pp. 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

1422 Laaber, C., Leitner, P., 2018. An evaluation of open-source software micro-
1423 benchmark suites for continuous performance assessment, in: Proceedings
1424 of the 15th International Conference on Mining Software Repositories (MSR
1425 '18). doi:10.1145/3196398.3196407.

1426 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking
1427 in the cloud. how bad is it really? Empirical Software Engineering 24.
1428 doi:10.1007/s10664-019-09681-1.

1429 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
1430 software fail? A case study and open problems, in: Proceedings of the 5th
1431 Asia-Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.26
1432 37237.

1433 Leitner, P., Cito, J., 2016. Patterns in the chaos—a study of performance
1434 variation and predictability in public IaaS clouds. ACM Transactions on
1435 Internet Technology 16. doi:10.1145/2885497.

1436 Liu, Y., Xu, C., Cheung, S.C., 2014. Characterizing and detecting performance
1437 bugs for smartphone applications, in: Proceedings of the 36th International
1438 Conference on Software Engineering (ICSE '14), pp. 1013–1024. doi:10.1
1439 145/2568225.2568229.

1440 Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee
1441 reproducibility. arXiv preprint arXiv:2601.12811.

1442 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing
1443 wrong data without doing anything obviously wrong!, in: Proceedings of the
1444 14th International Conference on Architectural Support for Programming
1445 Languages and Operating Systems (ASPLOS '09). doi:10.1145/1508244.
1446 1508275.

1447 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating
1448 the accuracy of Java profilers, in: Proceedings of the 31st ACM SIGPLAN
1449 Conference on Programming Language Design and Implementation (PLDI
1450 '10), pp. 187–197. doi:10.1145/1806596.1806618.

1451 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:
1452 Proceedings of the 38th International Conference on Software Engineering
1453 (ICSE '16). doi:10.1145/2884781.2884790.

1454 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang,
1455 Z., Yuan, X., 2020. Performance evaluation and modeling of cryptographic
1456 libraries for MPI communications. arXiv preprint arXiv:2010.06139.

1457 Nistor, A., Jiang, T., Tan, L., 2013a. Discovering, reporting, and fixing
1458 performance bugs, in: Proceedings of the 10th Working Conference on
1459 Mining Software Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.

1460 Nistor, A., Song, L., Marinov, D., Lu, S., 2013b. Toddler: Detecting perfor-
1461 mance problems via similar memory-access patterns, in: Proceedings of
1462 the 35th International Conference on Software Engineering (ICSE '13), pp.
1463 562–571. doi:10.1109/ICSE.2013.6606602.

1464 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
1465 optimization in Java. ACM Transactions on Programming Languages and
1466 Systems 28. doi:10.1145/1111596.1111598.

1467 de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception
1468 handling anti-patterns, in: Proceedings of the 25th International Conference
1469 on Program Comprehension (ICPC '17), pp. 328–331. doi:10.1109/ICPC
1470 .2017.1.

1471 Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting over-
1472 heads in the V8 JavaScript engine, in: 2021 IEEE International Symposium
1473 on Workload Characterization (IISWC). doi:10.1109/IISWC53511.2021.
1474 00013.

1475 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantar-
1476 cioglu, M., Yao, D.D., 2019. CryptoGuard: High precision detection of
1477 cryptographic vulnerabilities in massive-sized Java projects, in: Proceedings
1478 of the 2019 ACM SIGSAC Conference on Computer and Communications
1479 Security (CCS '19), pp. 2455–2472. doi:10.1145/3319535.3345659.

1480 Renwick, J., Spink, T., Franke, B., 2019. Low-cost deterministic C++ ex-
1481 ceptions for embedded systems, in: Proceedings of the 28th International
1482 Conference on Compiler Construction (CC 2019). doi:10.1145/3302516.
1483 3307346.

1484 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance
1485 profiler for optimizing the order of evaluations, in: Proceedings of the

1486 26th ACM SIGSOFT International Symposium on Software Testing and
1487 Analysis (ISSTA '17). doi:10.1145/3092703.3092716.

1488 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in
1489 JavaScript: An empirical study, in: Proceedings of the 38th International
1490 Conference on Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/
1491 2884781.2884829.

1492 Shatnawi, A., Al-Duwairi, B., Smarneh, A., 2024. Comprehensive empirical
1493 study of Python JWT libraries. *Procedia Computer Science* 238, 827–832.
1494 doi:10.1016/j.procs.2024.06.099.

1495 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current
1496 Practices. Request for Comments RFC 8725. Internet Engineering Task
1497 Force. doi:10.17487/RFC8725.

1498 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead
1499 of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the
1500 16th International Conference on Emerging Networking EXperiments and
1501 Technologies (CoNEXT '20). doi:10.1145/3386367.3431305.

1502 Song, L., Lu, S., 2014. Statistical debugging for real-world performance
1503 problems, in: Proceedings of the 2014 ACM International Conference
1504 on Object Oriented Programming Systems Languages & Applications
1505 (OOPSLA '14), pp. 561–578. doi:10.1145/2660193.2660234.

1506 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vul-
1507 nerabilities in JavaScript-based web servers, in: Proceedings of the 27th
1508 USENIX Security Symposium, pp. 361–376.

1509 Stefan, P., Horký, V., Bulej, L., Tůma, P., 2017. Unit testing performance in
1510 Java projects: Are we there yet?, in: Proceedings of the 8th ACM/SPEC
1511 International Conference on Performance Engineering (ICPE '17), pp. 401–
1512 412. doi:10.1145/3030207.3030226.

1513 Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective
1514 assessment of steady state performance in Java software: Are we there yet?
1515 *Empirical Software Engineering* 28. doi:10.1007/s10664-022-10247-x.

1516 Tuveri, N., Brumley, B.B., 2019. Start your ENGINes: Dynamically loadable
 1517 contemporary crypto, in: 2019 IEEE Cybersecurity Development (SecDev).
 1518 doi:10.1109/SecDev.2019.00014.

1519 Weimer, W., Necula, G.C., 2004. Finding and preventing run-time error
 1520 handling mistakes, in: Proceedings of the 19th Annual ACM SIGPLAN
 1521 Conference on Object-Oriented Programming, Systems, Languages, and
 1522 Applications (OOPSLA '04). doi:10.1145/1028976.1029011.

1523 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L.,
 1524 2023. JWTKey: Automatic cryptographic vulnerability detection in JWT
 1525 applications, in: Computer Security – ESORICS 2023, Springer. doi:10.1
 1526 007/978-3-031-51479-1_14.

1527 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W.,
 1528 Wang, B., 2026. Token time bomb: Evaluating JWT implementations for
 1529 vulnerability discovery, in: Proceedings of the Network and Distributed
 1530 System Security Symposium (NDSS '26).

1531 Zaman, S., Adams, B., Hassan, A.E., 2012. A qualitative study on performance
 1532 bugs, in: Proceedings of the 9th IEEE Working Conference on Mining
 1533 Software Repositories (MSR '12), pp. 199–208. doi:10.1109/MSR.2012.6
 1534 224281.

1535 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On
 1536 the relation between outdated Docker containers, severity vulnerabilities,
 1537 and bugs, in: Proceedings of the 26th IEEE International Conference on
 1538 Software Analysis, Evolution and Reengineering (SANER '19), pp. 491–501.
 1539 doi:10.1109/SANER.2019.8668013.

1540 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He,
 1541 P., Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting
 1542 overheads of service mesh sidecars, in: Proceedings of the 2023 ACM
 1543 Symposium on Cloud Computing (SoCC '23). doi:10.1145/3620678.3624
 1544 652.